Master Go, Vue, and Git

By Rebuilding the PEACE SME Grant Portal

A practical, chapter-by-chapter workbook generated from the application specification.

Master Go, Vue, and Git by Rebuilding the PEACE SME Grant Portal

This guide is a chapter-by-chapter book for converting the existing Flask-based PEACE SME Grant Portal into a Go backend and a Vue 3 frontend while learning Git through disciplined, real project work. It is written for a coder who is new to Go: every backend concept is introduced theoretically, then immediately attached to a feature from the portal.

The source of truth for the application behavior is Claude.md. Every chapter ties a technical concept to one part of the portal: authentication, applicant profiles, grants, admin reports, HFC fraud scoring, uploads, queues, caching, bilingual UI, and deployment.

Learning Goals

By the end of the book, you should be able to:

- Design and implement production Go HTTP services.
- Model PostgreSQL schemas and JSONB data in Go.
- Build secure authentication with JWT and bcrypt.
- Use middleware for auth, geo-blocking, access control, logging, and recovery.
- Implement Redis caching, concurrency controls, and background jobs.
- Integrate S3-compatible object storage and Brevo email delivery.
- Build a Vue 3 SPA using Composition API, Vue Router, Axios, Tailwind CSS, and Chart.js.
- Preserve an existing API contract while replacing the backend implementation.
- Use Git branches, commits, diffs, merges, tags, rebases, and pull requests as part of daily engineering work.

Recommended Repository Layout

The Go backend can evolve from the current minimal module into this structure:

```
cmd/server/
  main.go
internal/
  app/
  auth/
  cache/
  config/
  db/
  httpx/
  security/
  user/
  business/
  grant/
  admin/
  report/
  hfc/
  status/
  content/
  storage/
  mail/
migrations/
frontend/
  src/
```

Chapters

Use the Concept Index when you want to study by topic instead of chapter order.

1. Read the Existing System Like an Engineer
2. Git Foundations for a Rewrite
3. Go Project Structure and HTTP Server
4. Configuration, Environment, and Application Toggles
5. PostgreSQL, Migrations, and Data Modeling
6. Authentication, JWT, bcrypt, and Middleware
7. Applicant Registration, Login, and Business Profiles
8. Grant Applications and Workflow Rules
9. Uploads, S3-Compatible Storage, and Media References
10. Redis, Caching, Access Slots, and Background Jobs
11. Admin, Reports, CSV, and Database Browser
12. HFC Fraud Detection and Approval Authority
13. Vue 3 Frontend Foundations
14. Vue Applicant and Admin Interfaces
15. Testing, Debugging, and API Compatibility
16. Deployment, Release Git, and Mastery Roadmap

1. Go Beginner Primer Through Portal Features
2. Project Parallel Workbook
3. Vue and Git Practice Lab

How to Work Through the Book

Use one branch per chapter:

```
git checkout -b chapter-03-go-server
```

Make small commits:

```
git add .
git commit -m "Add Go server bootstrap"
```

At the end of each chapter, compare your work against the contract in Claude.md. The goal is not merely to make something work. The goal is to reproduce the same API paths, request shapes, response shapes, business rules, and frontend behavior with a Go + Vue implementation you understand deeply.

The Learning Pattern Used in Every Chapter

Each chapter should be read in this order:

1. Theory: learn the language or engineering concept.
2. Application parallel: identify where the PEACE SME portal needs that concept.
3. Backend implementation: build the Go package, handler, service, repository, or worker piece.
4. Frontend implementation: connect the Vue screen or component to the matching API behavior.
5. Git practice: save the work as a small, reviewable commit.
6. Mastery check: prove you can explain and modify the concept without copying code.

For example, you do not learn Go structs in isolation. You learn structs by modeling `users`, `businesses`, and `grants`. You do not learn Vue reactivity in isolation. You learn reactivity by building the grant form where changing

contribution type reveals cash or in-kind fields. You do not learn Git branches abstractly. You learn branches by creating `feature/grant-access-whitelist`, reviewing its diff, and merging it after tests pass.

Concept Index

This index maps the major skills to the chapters where they are practiced.

Go

Concept	Chapters
Project layout	3
`cmd` and `internal` packages	3
HTTP handlers	3, 7, 8, 11
JSON request/response helpers	3
Configuration loading	4
Environment toggles	4, 7, 8, 10, 12
Interfaces	8, 10, 12
Dependency injection	3, 7
Context propagation	5, 6
Error handling	3, 6, 15
Middleware	6
JWT	6
bcrypt	6
Repository pattern	5
Transactions	5, 12
Background worker boundaries	10, 12
Testing	15
Beginner Go concept map	17
Feature-by-feature Go practice	18

PostgreSQL

Concept	Chapters
Schema preservation	5
Migrations	5, 16
Foreign keys and cascade delete	5
Unique constraints	7, 8
JSONB	5, 8
Nullable columns	5
Indexes	5, 11
Paginated queries	11
Filtered reports	11
Defensive dynamic SQL	11
Audit tables	12
Feature-to-table mapping	18

Vue

Concept	Chapters
Vue 3 SPA structure	13
Composition API	13, 14
`ref`, `reactive`, `computed`, `watch`	13, 14
Vue Router	13
Route guards	13
Axios clients	13
localStorage token handling	13
Bilingual English/Urdu UI	13, 14
RTL layout	13, 14
Large forms	14
Repeating form rows	14
Admin tables	14
Charts	14
Loading/error/empty states	14
Vue practice labs	19
Concept-to-screen mapping	19

Git

Concept	Chapters
Status and diffs	1, 2
Branches	2
Small commits	2
Staging area	2
Commit messages	2
Reading history	2
Merge	2
Rebase	2
Conflict resolution	2
Tags	16
Release branches	16
Hotfix branches	16
Bisect	15
Feature branch labs	19
Commit review workflow	19

Security

Concept	Chapters
Password verification	6

Concept	Chapters
Bearer auth	6, 13
Admin roles	6, 12
Approver-only endpoints	6, 12
Geo-blocking	6, 10
Access slots	10
Upload ownership checks	9
Query token exports	11
SQL injection prevention	11
End-to-end auth boundaries	18, 19

Production Engineering

Concept	Chapters
API compatibility	1, 15
Caching	10, 11
Redis TTLs	10
S3-compatible uploads	9
Email jobs	10, 12
HFC scoring	12
Auditability	12
Docker builds	16
Migrations at deploy time	16
Rollback planning	16
Release tagging	16
End-to-end project milestones	18
Full capstone workflow	19

Chapter 1: Read the Existing System Like an Engineer

Purpose

Before writing Go or Vue code, learn to extract behavior from an existing application. The PEACE SME Grant Portal is not just a set of pages. It is a workflow:

1. Applicant registration and login.
2. Business profile entry.
3. Document and media uploads.
4. Admin eligibility decisions.
5. Whitelisted grant application submission.
6. HFC fraud scoring.
7. Approval by an approving authority.
8. Notifications, reports, exports, FAQs, updates, and operational controls.

The rewrite succeeds only if these behaviors remain compatible.

For a beginner, this chapter teaches an important professional habit: do not start a rewrite by opening an editor and inventing code. Start by reading the system until you can describe what must remain true after the rewrite.

Theory: What Is a Rewrite?

A rewrite is not "build a new app that looks similar." A rewrite is a controlled replacement of implementation while preserving required behavior.

In this project:

- Old implementation: Flask backend plus Vue frontend.
- New implementation: Go backend plus Vue 3 frontend.
- Stable contract: PostgreSQL schema, API paths, JSON shapes, auth behavior, business rules, and user workflows.

Think of the rewrite as changing the engine of a running system. The outside controls should still work. The frontend expects the same endpoints. The database stores the same business facts. Admins expect the same reports. Applicants expect the same workflow.

Application Parallel: The First Task You Are Really Building

Your first project task is not a route or a page. It is a map.

Build a map that answers:

- Which frontend view calls which endpoint?
- Which endpoint uses which service?
- Which service reads or writes which table?
- Which business rule protects that operation?
- Which Git branch will contain that work?

Example:

User action	Vue view	API endpoint	Go package	Tables	Rule
Applicant logs in	`UserLogin.vue`	`POST /api/login`	`internal/user`, `internal/auth`	`users`	blocked users cannot login
Applicantsaves business	`SmeBusinessProfile.vue`	`POST /api/business`	`internal/business`	`businesses`	one business per user
Admin approves grant	`AdminGrantDetail.vue`	`POST /api/admin/grants/<user_id>/approve`	`internal/grant`, `internal/admin`	`grants`, `grant_approval_logs`	approver role required

This map becomes your implementation checklist.

What to Read

Start with:

- `Claude.md`
- `README.md`
- `go.mod`
- `cmd/server/main.go`

The current Go service is only a health-check stub. That is useful: the project is clean enough to grow in deliberate layers.

System Map

The original application has these major boundaries:

Boundary	Flask module	Go package to create	Vue area
Auth	`auth_service.py`, `security.py`	`internal/auth`, `internal/security`	Login views, route guards
Users	`user_service.py`	`internal/user`	Registration, login, dashboard
Business profile	`user_service.py`	`internal/business`	`SmeBusinessProfile.vue`
Grants	`grant_service.py`	`internal/grant`	`SmeGrantApplication.vue`
Reports	`report_service.py`	`internal/report`	Admin report views
Admin	`admin_service.py`	`internal/admin`	Admin dashboard and tools
HFC	`hfc_service.py`, `hfc_admin_service.py`	`internal/hfc`	HFC admin views
Applicant status	`status_service.py`	`internal/status`	Applicant status admin view
Content	updates, FAQs	`internal/content`	Landing page, FAQ bot
Storage	`s3_service.py`	`internal/storage`	Upload components
Mail	`mail_service.py`	`internal/mail`	Approval side effects

API Contract Thinking

The Vue frontend depends on:

- Exact endpoint paths under `/api`.
- Exact localStorage keys: `userToken`, `adminToken`, `language`.

- Exact pagination shapes such as `{data, total, page, per_page}`.
- JWTs sent as `Authorization: Bearer <token>`.
- Admin JWT fields such as `is_admin` and `is_approver`.
- English stored values even when the UI is Urdu.

When converting Flask to Go, you are not free to invent a cleaner public contract until you also migrate the frontend.

Contract vs Implementation

The contract is what other parts of the system depend on. The implementation is how you satisfy it.

Contract	Implementation detail
`POST /api/login` returns `{token, user_id, language}`	Go may use `net/http`, `chi`, or another router
Passwords use bcrypt	Repository structure is up to you
JWT uses HS256 and `JWT_SECRET_KEY`	Token service can be written as a small package
Reports return `{data, total, page, per_page}`	SQL can be optimized later
`GRANT_REQUIRE_SELECTION=1` gates grant submission	The check can live in `GrantService`

As a learner, this distinction prevents a common mistake: changing the public behavior while trying to improve the internal design.

Beginner Go Lens

This system will teach Go in layers:

1. Structs model data such as users and grants.
2. Functions and methods implement operations such as login and approval.
3. Interfaces separate side effects such as S3 and email from business logic.
4. Errors make invalid states explicit.
5. Context carries request cancellation and identity.
6. Goroutines and workers handle slow work such as email and HFC scoring.

Do not try to master all Go features before building. Learn each feature when the application gives you a reason to use it.

Exercise

Create a file named `guide/app-contract-notes.md` while studying. Capture:

- Every endpoint you plan to implement.
- Every table you plan to migrate.
- Every feature toggle.
- Every auth rule.
- Every response shape that the frontend depends on.

Add one more section called "parallel learning notes." For every feature, write the Go concept it teaches:

Feature	Go concept	Vue concept	Git concept
Login	structs, bcrypt, JWT, errors	form state, localStorage	small auth branch
Business profile	validation, repository methods	reactive form	commit create and update separately

Feature	Go concept	Vue concept	Git concept
Reports	SQL filters, maps, pagination	table state, query params	review staged diff carefully

Git Practice

Initialize your learning branch:

```
git status
git checkout -b chapter-01-system-reading
git add guide
git commit -m "Add system reading notes"
```

Use `git diff --staged` before committing. This teaches you to review your own work before Git records it.

Mastery Check

You understand this chapter when you can explain:

- Why the Go rewrite must preserve the Flask API contract.
- Which modules belong in backend packages.
- Which parts of the frontend are public, applicant-only, and admin-only.
- Why `Claude.md` is a behavioral specification, not just documentation.

Chapter 2: Git Foundations for a Rewrite

Purpose

Git is not separate from the rewrite. It is the safety system that lets you change a large application without losing track of intent.

Theoretical Background

Git as a Directed Acyclic Graph (DAG)

Git stores history not as a list of changes, but as a Directed Acyclic Graph (DAG) of commit objects. Each commit contains:

- A cryptographic SHA-1 hash (identifying the commit uniquely based on file contents, metadata, parent hashes, author, and timestamp).
- A pointer to zero or more parent commits.
- A snapshot of the entire directory tree (not delta changes).

Understanding this graph structure helps you visualize operations:

- A **Branch** is simply a mutable pointer to a specific commit.
- **HEAD** is a pointer to the currently checked-out commit or branch.

The Three-State Architecture

Git projects consist of three primary zones:

1. **Working Directory (Workspace):** The local sandbox where you edit files on disk.
2. **Staging Area (Index):** A binary file tracking the changes prepared for the next commit. This acts as a preparation area.
3. **Commit History (Repository/Git Directory):** The permanent metadata database storing the commit graph (`.git/objects/`).

```
[ Working Directory ] --- git add ---> [ Staging Area (Index) ] --- git commit ---> [ Commit
History (.git) ]
[                  ] <--- checkout --- [                    ] <--- commit -------- [
              ]
```

Merging vs. Rebasing

- **Merge (`git merge`):** Integrates two branches by creating a special "merge commit" that has two parent commits. This preserves the exact historical timeline and order of changes, but can make the history cluttered with merge commits.
- **Rebase (`git rebase`):** Rewrites commit history by moving the base of your branch to a new starting point. It applies your commits sequentially as new commits on top of the target branch. This creates a clean, linear history but alters the SHA-1 hashes of the original commits.

[!WARNING] Never rebase commits that have been pushed to a shared public repository. Doing so rewrites history that others depend on.

External Resources

- Git Book - Git Internals
- Git Book - Git Branching and Rebasing

Core Workflow

Use this loop for every chapter:

```
git status
git checkout -b chapter-XX-short-name
git diff
git add <files>
git diff --staged
git commit -m "Describe the completed behavior"
```

Good commits are small, reviewable, and named after behavior:

- `Add Go configuration loader`
- `Implement user login endpoint`
- `Add Vue admin route guards`
- `Add HFC scoring tests`

Avoid vague messages:

- `updates`
- `fix`
- `changes`

Practical Examples

Example 1: Creating and Merging a Feature Branch

To start a new feature and safely integrate it back into the main branch:

```
# 1. Ensure you are on main and up to date
git checkout main
git pull origin main

# 2. Create and switch to the new feature branch
git checkout -b feature/auth-service

# 3. (After making changes to internal/auth/auth.go)
# Check what has changed in the working directory
git status

# 4. Stage your changes
git add internal/auth/auth.go

# 5. Review the staged diff before committing
git diff --staged

# 6. Commit the staged changes
git commit -m "Implement authentication service skeleton"

# 7. Merge the changes back into main
git checkout main
git merge feature/auth-service
```

Example 2: Interactive Rebase to Clean Up Commits

Before submitting a pull request, you may want to squash minor "fixup" commits into a single logical commit:

```
# Start an interactive rebase for the last 3 commits
git rebase -i HEAD~3
```

This will open an editor showing:

```
pick a1b2c3d Add user registration validator
pick d4e5f6g Fix typo in registration validator
pick h7i8j9k Add tests for registration validation
```

To squash the typo fix into the main commit, change the file to:

```
pick a1b2c3d Add user registration validator
squash d4e5f6g Fix typo in registration validator
pick h7i8j9k Add tests for registration validation
```

Save and close the editor. Git will combine the first two commits and prompt you for a new commit message.

Example 3: Resolving a Merge Conflict

If you and another developer edit the same line of a file, Git will report a conflict:

```
Auto-merging cmd/server/main.go
CONFLICT (content): Merge conflict in cmd/server/main.go
Automatic merge failed; fix conflicts and then commit the result.
```

Opening `cmd/server/main.go` will show the conflict markers:

```
<<<<<<< HEAD
    addr := ":8080"
=======
    addr := ":9000"
>>>>>>> feature/custom-port
```

To resolve this conflict:

1. Discuss or decide which value is correct (or combine them).
2. Delete the conflict markers (<<<<<<<, =======, >>>>>>>) and edit the code to the desired state:

   ```
   addr := ":9000"
   ```

1. Stage the resolved file:

   ```
   git add cmd/server/main.go
   ```

1. Complete the merge commit:

   ```
   git commit -m "Resolve merge conflict, standardizing on port 9000"
   ```

Reading History

Use:

```
git log --oneline --decorate --graph --all
git show <commit>
git diff main...HEAD
```

`git show` answers "what changed in this commit?" `git diff main...HEAD` answers "what does this branch change compared with main?"

Handling Mistakes

Unstage a file:

```
git restore --staged path/to/file
```

Discard changes in one file only when you are sure:

```
git restore path/to/file
```

Inspect before undoing:

```
git diff path/to/file
```

Mastery Check

You understand this chapter when you can:

- Create a branch per chapter.
- Review staged changes before committing.
- Explain the difference between working tree, staging area, and commit history.
- Recover from accidental staging.
- Resolve a simple conflict without panic.

Chapter 3: Go Project Structure and HTTP Server

Purpose

Replace the stub server with a production-shaped Go application. In this chapter, we will learn the basics of Go's dependency management, package visibility, custom data types, pointers, handlers, and application assembly. We will apply these concepts by building the shell that will eventually host every PEACE SME API endpoint.

This chapter should answer a beginner question: "Where does code go in a Go web app?"

Foundational Concepts Explained Simply

1. Go Modules (`go mod`)

In Go, a **Module** is a collection of Go packages stored in a file tree with a `go.mod` file at its root. The `go.mod` file defines the module's import path (its name) and lists the third-party dependency packages required to build the project.

- **Initialization:** Running `go mod init peace-sme-go` creates the `go.mod` file.
- **Adding Dependencies:** When you import a third-party library (like a router or database driver) and run `go get github.com/jackc/pgx/v5`, Go downloads the package and adds it as a dependency in `go.mod`.
- `go.sum` **File:** Go automatically generates a `go.sum` file. This file contains the cryptographic hashes of the dependencies, ensuring that future builds download the exact same code and have not been tampered with.

Application parallel: the portal backend will need dependencies for PostgreSQL, Redis, JWTs, bcrypt, and S3. Go records those dependencies in `go.mod`, which makes the backend reproducible for you, CI, and deployment.

2. Packages and Project Organization

Every Go file must start with a package declaration (e.g., `package main` or `package config`).

- **Visibility (Exporting):** Go uses a simple rule for access control:
- If a struct, function, variable, or field name starts with a **Capital Letter** (e.g., `UserID`, `FindUser()`), it is **exported** (public) and can be accessed by other packages.
- If it starts with a **lowercase letter** (e.g., `userID`, `findUser()`), it is **unexported** (private) and can only be accessed within the same package.
- **Structure Convention:**
- `cmd/`**:** Holds entry points (main packages) that compile into executables.
- `internal/`**:** Packages containing business logic. Go enforces a compiler rule: packages inside `internal/` cannot be imported by external projects outside this module, protecting your internal implementation details.

Application parallel: `internal/grant` should be importable by your server, but not by some unrelated external project. The portal's business rules are internal product logic, not a public Go library.

2.1 A Beginner-Friendly Package Rule

Start with packages named after responsibilities, not technical layers only:

```
internal/user       applicant login and profile behavior
internal/business   business profile behavior
internal/grant      grant application and approval behavior
internal/report     admin report queries
internal/hfc        fraud scoring behavior
```

Inside each package, you can still separate handlers, services, repositories, and models:

```
internal/grant/
  handler.go
```

```
service.go
repository.go
model.go
validation.go
```

This layout helps you read one feature vertically.

3. Structs (Custom Data Types)

A **Struct** is a typed collection of fields. It is used to group related data together to form custom data structures (like a User profile or a Business record).

```
// Defining a Struct
type Book struct {
    Title  string // Exported field
    Pages  int    // Exported field
    author string // Unexported (private) field
}
```

Struct Tags

Fields in a struct can have **tags** (strings metadata) attached to them. Tags are commonly used by encoders to map struct fields to other formats like JSON or SQL database columns:

```
type User struct {
    Email string `json:"email_address"` // When serialized to JSON, this key becomes
"email_address"
}
```

4. Pointers (Memory Management)

When you pass a variable to a function in Go, Go always creates a **copy** of that variable's value.

- If you pass a large struct, copying it consumes memory.
- If you modify the struct inside the function, you are modifying the *copy*, not the original struct.

Pointers solve this. A pointer stores the **memory address** of a variable rather than its value.

- **Address-of Operator (&):** Placing an ampersand before a variable gets its pointer (memory address).
- **Dereferencing Operator (*):** Placing an asterisk before a pointer variable accesses the underlying value stored at that memory address.
- **Type Declaration (asterisk-Type):** the notation `*User` means "a pointer to a User struct".

```
package main

import "fmt"

type Counter struct {
    Val int
}

// Pass by value (copies the struct)
func incrementValue(c Counter) {
    c.Val++ // Modifies only the copy
}

// Pass by pointer (receives the memory address)
func incrementPointer(c *Counter) {
    c.Val++ // Modifies the original struct
}

func main() {
    c := Counter{Val: 10}

    incrementValue(c)
    fmt.Println(c.Val) // Prints 10 (original was unchanged)

    incrementPointer(&c) // Pass the memory address
    fmt.Println(c.Val) // Prints 11 (original was modified directly)
}
```

Application parallel: when you create an HTTP handler, it often needs access to a service. You usually store a pointer:

```
type Handler struct {
    service *Service
}
```

Each request uses the same handler instance and the same service dependencies. You do not copy the service for every request.

5. HTTP Handlers

In Go's standard library, a handler is any value that can serve an HTTP request:

```
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}
```

Most beginner code starts with:

```
func healthHandler(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("ok"))
}
```

For the portal, move toward method handlers:

```
func (h *Handler) Login(w http.ResponseWriter, r *http.Request) {
    // decode JSON, call service, write response
}
```

Application parallel: every API endpoint in `Claude.md` eventually becomes one handler method.

External Resources

- Go Dev: Tutorial on Go Modules
- A Tour of Go: Structs
- A Tour of Go: Pointers

Project Implementation: Build the Server Skeleton First

The first backend milestone should be small:

```
cmd/server/main.go
internal/app/app.go
internal/config/config.go
internal/httpx/json.go
internal/health/handler.go
```

`main.go` Should Stay Boring

`main.go` should not contain grant rules, SQL strings, JWT parsing, or S3 logic. It should compose the program:

```
func main() {
    cfg, err := config.Load()
    if err != nil {
        log.Fatal(err)
    }

    app, err := app.New(cfg)
    if err != nil {
        log.Fatal(err)
    }

    if err := app.Run(context.Background()); err != nil {
        log.Fatal(err)
    }
}
```

Application parallel: when you later add `/api/grant`, `main.go` should barely change. You add a grant package and register its routes inside the app assembly.

JSON Helpers

Most PEACE SME endpoints return JSON. Add one helper early:

```
func WriteJSON(w http.ResponseWriter, status int, value any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(value)
}
```

And one error helper:

```
func WriteError(w http.ResponseWriter, status int, code string, message string) {
    WriteJSON(w, status, map[string]string{
        "error": code,
        "message": message,
    })
}
```

Application parallel: login, business profile, grant submission, and admin reports should all use the same response-writing helpers.

Project Implementation: Defining the Application Models (Structs)

Now we will create the core models for the PEACE SME Grant Portal. We will write them in a centralized location first, so you can practice struct layouts, field types, nullable values, and JSON tags.

Create a file named internal/db/models.go to declare all structs.

1. Database Nullable Types

In PostgreSQL, fields can be `NULL`. Standard Go primitive types like `string` or `int` cannot hold `nil`.

- To scan nullable database columns safely, Go provides database wrappers under the standard `database/sql` package: `sql.NullString`, `sql.NullInt64`, `sql.NullBool`, `sql.NullTime`.
- When using these, access the value via `.String` or `.Int64` after checking if `.Valid` is true.

2. Complete Models File Boilerplate

```
// File: internal/db/models.go
package db

import (
■"database/sql"
■"encoding/json"
■"time"
)

// User represents a registered applicant or admin user.
type User struct {
■UserID            int64          `json:"user_id"`
■EmailAddress      string         `json:"email_address"`
■HashedPassword    string         `json:"-"` // "-" prevents password hash from rendering in
JSON
■FirstName         sql.NullString `json:"first_name"`
■LastName          sql.NullString `json:"last_name"`
■MiddleName        sql.NullString `json:"middle_name"`
■CNIC              sql.NullString `json:"cnic"`
■Language          sql.NullString `json:"language"`
■Gender            sql.NullString `json:"gender"`
■MobileNo          sql.NullString `json:"mobile_no"`
■WhatsappNumber    sql.NullString `json:"whatsapp_number"`
■TermsAccepted     bool           `json:"terms_accepted"`
■Status            string         `json:"status"` // 'blocked' | 'unblocked'
■LastLoginIP       sql.NullString `json:"last_login_ip"`
■DeviceFingerprint sql.NullString `json:"device_fingerprint"`
■CreatedAt         time.Time      `json:"created_at"`
}

// Business represents the applicant's business profile.
type Business struct {
■BusinessID                  int64           `json:"business_id"`
```

```
■UserID                       int64           `json:"user_id"`
■NameOfBusiness               sql.NullString  `json:"name_of_business"`
■BusinessRegistrationNumber   sql.NullString  `json:"business_registration_number"`
■BusinessRegistrationDate     sql.NullTime    `json:"business_registration_date"`
■BusinessRegistrationAuthority json.RawMessage `json:"business_registration_authority"` //
JSONB Array
■OtherAuthorityText           sql.NullString  `json:"other_authority_text"`
■BusinessFullAddress          sql.NullString  `json:"business_full_address"`
■SocialMediaPage              sql.NullString  `json:"social_media_page"`
■SocialMediaPage2             sql.NullString  `json:"social_media_page_2"`
■SocialMediaPage3             sql.NullString  `json:"social_media_page_3"`
■SocialMediaPage4             sql.NullString  `json:"social_media_page_4"`
■MaleEmployees                sql.NullInt64   `json:"male_employees"`
■FemaleEmployees              sql.NullInt64   `json:"female_employees"`
■BusinessLocationDistrict     sql.NullString  `json:"business_location_district"`
■BusinessSector               sql.NullString  `json:"business_sector"`
■HowDidYouHear                sql.NullString  `json:"how_did_you_hear"`
■HasSRSPRelation              bool            `json:"has_srsp_relation"`
■SRSPRelativesData            json.RawMessage `json:"srsp_relatives_data"` // JSONB Array
■CreatedAt                    time.Time       `json:"created_at"`
}

// BusinessDocument represents uploaded files (e.g., CNIC scans, bank statements).
type BusinessDocument struct {
■DocumentID   int64     `json:"document_id"`
■BusinessID   int64     `json:"business_id"`
■DocumentType string    `json:"document_type"`
■FileName     string    `json:"file_name"`
■FilePath     string    `json:"file_path"` // Public S3 URL
■MIMEType     string    `json:"mime_type"`
■CreatedAt    time.Time `json:"created_at"`
}

// FinancedItem represents an item requested within a grant application.
type FinancedItem struct {
■Item          string  `json:"item"`
■Quantity      int     `json:"quantity"`
■EstimatedCost float64 `json:"estimated_cost"`
}

// SRSPRelative represents a relative working at SRSP.
type SRSPRelative struct {
■Name     string `json:"name"`
■Position string `json:"position"`
■Office   string `json:"office"`
}

// Grant represents the full grant application form.
type Grant struct {
■GrantID                    int64           `json:"grant_id"`
■UserID                     int64           `json:"user_id"`
■ExpressionOfInterest       sql.NullString  `json:"expression_of_interest"` // JSON string
array
■GrantRequired              sql.NullFloat64 `json:"grant_required"`
■ApplicationDate            sql.NullTime    `json:"application_date"`
■Status                     string          `json:"status"` // 'Pending', 'Approved', etc.
■ContributionType           sql.NullString  `json:"contribution_type"`
■FinancialAmount            sql.NullFloat64 `json:"financial_amount"`
■FinancialAmountWords       sql.NullString  `json:"financial_amount_words"`
■InKindDetails              sql.NullString  `json:"inkind_details"`
■InKindValue                sql.NullFloat64 `json:"inkind_value"`
■ContributionUtilization    sql.NullString  `json:"contribution_utilization"`
■GrantSupportGrowth         sql.NullString  `json:"grant_support_growth"`
■JobCreationDetails         sql.NullString  `json:"job_creation_details"`
■GrantAmountWords           sql.NullString  `json:"grant_amount_words"`
■OtherPurposeText           sql.NullString  `json:"other_purpose_text"`
■HowDidYouHear              sql.NullString  `json:"how_did_you_hear"`
■ApprovedAmount             sql.NullFloat64 `json:"approved_amount"`
■ApprovalReason             sql.NullString  `json:"approval_reason"`
■ApprovedAt                 sql.NullTime    `json:"approved_at"`
■ApprovedBy                 sql.NullString  `json:"approved_by"`
■HFCStatus                  string          `json:"hfc_status"` // 'HFC_Pending', 'Clear', etc.
■HFCScore                   int             `json:"hfc_score"`
■HFCRiskLevel               string          `json:"hfc_risk_level"`
■HFCLastEvaluatedAt         sql.NullTime    `json:"hfc_last_evaluated_at"`
■HFCModelVersion            sql.NullString  `json:"hfc_model_version"`
■DomicileDistrict           sql.NullString  `json:"domicile_district"`
```

```
■BusinessType               json.RawMessage `json:"business_type"`
■BusinessTypeOther          sql.NullString  `json:"business_type_other"`
■TaxRegistrationStatus      json.RawMessage `json:"tax_registration_status"`
■NTNRegistrationNo          sql.NullString  `json:"ntn_registration_no"`
■TaxFilerStatus             sql.NullString  `json:"tax_filer_status"`
■WorkingCapital             bool            `json:"working_capital"`
■FinancedItems              json.RawMessage `json:"financed_items"` // Scans nested
FinancedItem slice
■ExpectedProductionIncrease sql.NullString  `json:"expected_production_increase"`
■EmploymentGrid             json.RawMessage `json:"employment_grid"`
■DeclarationAccepted        bool            `json:"declaration_accepted"`
■DeclarationName            sql.NullString  `json:"declaration_name"`
■HasSRSPRelative            bool            `json:"has_srsp_relative"`
■SRSPRelatives              json.RawMessage `json:"srsp_relatives"` // Scans nested
SRSPRelative slice
}

// HFCEvaluation holds audit records of run fraud scoring rules.
type HFCEvaluation struct {
■EvaluationID int64     `json:"evaluation_id"`
■UserID       int64     `json:"user_id"`
■Score        int       `json:"score"`
■RiskLevel    string    `json:"risk_level"`
■RuleDetails  string    `json:"rule_details"` // JSON string breakdown
■EvaluatedAt  time.Time `json:"evaluated_at"`
}
```

Target Shape

```
cmd/server/main.go
internal/app/app.go
internal/config/config.go
internal/httpx/json.go
internal/health/handler.go
internal/db/models.go
```

`main.go` should do very little:

1. Load config.
2. Build the app.
3. Start the HTTP server.
4. Shut down cleanly on signals.

Handler Style

Prefer explicit handlers:

```
func (h *Handler) Health(w http.ResponseWriter, r *http.Request) {
    httpx.WriteJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

This style keeps framework magic low while learning Go.

Practical Examples

Example: Creating a Bootstrappable Main Package

We compile our server bootstrapping code inside `/cmd/server/main.go`. In the example below, we initialize a server struct passing it by pointer:

```
// File: cmd/server/main.go
package main

import (
■"log"
```

```
■"net/http"
■"time"
)

type Server struct {
■Addr string
}

// NewServer allocates a server struct and returns a pointer to it.
func NewServer(addr string) *Server {
■return &Server{Addr: addr}
}

func (s *Server) Start() error {
■mux := http.NewServeMux()
■mux.HandleFunc("/api/health", func(w http.ResponseWriter, r *http.Request) {
■■w.Write([]byte(`{"status":"healthy"}`))
■})

■srv := &http.Server{
■■Addr:         s.Addr,
■■Handler:      mux,
■■ReadTimeout:  5 * time.Second,
■■WriteTimeout: 5 * time.Second,
■}

■log.Printf("Bootstrapping server on %s", s.Addr)
■return srv.ListenAndServe()
}

func main() {
■// Allocate server using pointer construct
■srv := NewServer(":8080")
■if err := srv.Start(); err != nil {
■■log.Fatalf("Server startup failed: %v", err)
■}
}
```

Mastery Check

You understand this chapter when you can:

- Explain why we compile executables in `cmd/` and put logic in `internal/`.
- Initialize a Go module and fetch libraries.
- Define a custom struct with JSON tags.
- Explain the difference between passing a struct by value and passing it by pointer.
- Use `sql.NullString` to parse nullable database fields.

Chapter 4: Configuration, Environment, and Application Toggles

Purpose

The Flask application is controlled heavily by environment variables. The Go rewrite must preserve that behavior. In this chapter, we will learn how Go handles errors as normal values, how to use dynamic lists (slices), and key-value tables (maps). We will then implement a robust configuration parser that processes application toggles and administrator user lists.

Foundational Concepts Explained Simply

1. Errors as Values

In Go, there are no `try/catch` exception blocks. Instead, errors are treated as normal values that implement the standard `error` interface (which requires a single `Error() string` method).

- **Handling Errors:** If a function can fail, it returns an error as its last return parameter. The caller must explicitly check if the error is not `nil`:

```
file, err := os.Open("config.json")
if err != nil {
    // Handle the error (fail fast, log it, or return it)
    return err
}
```

- **Error Wrapping:** When returning an error from a function, it is best practice to add context to it using the `%w` verb in `fmt.Errorf`. This preserves the original error for downstream investigation:

```
if err != nil {
    return fmt.Errorf("failed to open config: %w", err)
}
```

2. Slices (Dynamic Arrays)

Unlike standard arrays in Go which have a fixed size defined at compile-time, a **Slice** is a dynamically-sized view into an underlying array.

- **Declaration & Appending:** Use the `append` function to dynamically add elements to a slice:

```
var list []string // Declares a nil slice
list = append(list, "admin1")
list = append(list, "admin2")
fmt.Println(list) // Prints ["admin1", "admin2"]
```

- **Make:** You can pre-allocate a slice with a specified capacity using `make([]Type, length, capacity)` to optimize memory allocations.

3. Maps (Key-Value Tables)

A **Map** is an unordered collection of key-value pairs (a hash map).

- **Initialization:** Maps must be initialized using `make` before you can write to them, otherwise it will cause a panic:

```
// Creating a map mapping country codes (strings) to existence checks (booleans)
countryMap := make(map[string]bool)
countryMap["PK"] = true
countryMap["AE"] = true
```

- **The Comma-OK Idiom:** To check if a key exists in a map without raising an error:

```
exists, ok := countryMap["PK"]
if ok {
    fmt.Println("PK is allowed:", exists)
```

```
} else {
    fmt.Println("PK is not in the map")
}
```

External Resources

- Go Blog: Go Video on Error Handling
- A Tour of Go: Slices
- A Tour of Go: Maps

Required Configuration Areas

Model these as typed Go fields:

Area	Variables
Auth	`JWT_SECRET_KEY`, `ADMIN_USERS_JSON`, `ADMIN_USERS_PLAIN_JSON`
Database	`POSTGRES_HOST`, `POSTGRES_PORT`, `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, pool sizes
Redis	Redis URL or host/port
S3	endpoint, access key, secret key, bucket, public base URL, ACL
Email	Brevo key, sender email, sender name, approval notification email
Toggles	`GRANT_APPLICATION_OPEN`, `GRANT_REQUIRE_SELECTION`, `HFC_SHADOW_MODE`, `HFC_ASYNC_ENABLED`
Access	`ACCESS_CONTROL_ENABLED`, `MAX_ACTIVE_APPLICANTS`, `ACCESS_SLOT_TTL_SEC`, `GEO_BLOCK_ENABLED`, `ALLOWED_COUNTRY_CODES`
Cache	`CACHE_ENABLED`, `CACHE_PREFIX`, TTL values

Typed Config

Do not pass raw environment strings throughout the app. Convert once:

```
type Config struct {
    HTTPAddr              string
    JWTSecret             string
    GrantApplicationOpen  bool
    GrantRequireSelection bool
    HFCShadowMode         bool
    CachePrefix           string
    AllowedCountryCodes   map[string]bool // Map lookup for allowed countries
}
```

Typed config prevents repeated string comparisons such as `os.Getenv("X") == "1"` across the codebase.

Admin Users

`ADMIN_USERS_JSON` contains:

```
[
  {
    "username": "admin1",
    "password_hash": "$2b$12$...",
    "role": "admin",
    "can_approve_grants": false
  }
]
```

In Go, unmarshal it into:

```
type AdminUser struct {
    Username         string `json:"username"`
    PasswordHash     string `json:"password_hash"`
    Role             string `json:"role"`
    CanApproveGrants bool   `json:"can_approve_grants"`
}
```

Practical Examples

Example: Writing a Fail-Fast Configuration Loader

This complete configuration loader parses basic string flags, converts strings to integers/booleans, maps country codes into a lookup Map, and unmarshals the admin JSON array into a Slice:

```
// File: internal/config/config.go
package config

import (
■"encoding/json"
■"fmt"
■"os"
■"strconv"
■"strings"
)

type AdminUser struct {
■Username         string `json:"username"`
■PasswordHash     string `json:"password_hash"`
■Role             string `json:"role"`
■CanApproveGrants bool   `json:"can_approve_grants"`
}

type Config struct {
■Port                 int
■JWTSecret            string
■GrantApplicationOpen bool
■AllowedCountryCodes  map[string]bool // Fast lookup map
■AdminUsers           []AdminUser     // Slice of admin users
}

// Load compiles config values, returning wrapping errors on parsing failures.
func Load() (*Config, error) {
■cfg := &Config{
■■AllowedCountryCodes: make(map[string]bool),
■}

■// 1. Parsing Port integer
■portStr := getEnv("PORT", "8080")
■port, err := strconv.Atoi(portStr)
■if err != nil {
■■return nil, fmt.Errorf("parsing PORT %q failed: %w", portStr, err)
■}
■cfg.Port = port

■// 2. Validate required secret
■cfg.JWTSecret = os.Getenv("JWT_SECRET_KEY")
■if cfg.JWTSecret == "" {
■■return nil, fmt.Errorf("required environment key JWT_SECRET_KEY is empty")
■}

■// 3. Parsing application toggle bool
■cfg.GrantApplicationOpen = getEnv("GRANT_APPLICATION_OPEN", "0") == "1"

■// 4. Split allowed country codes string into a fast lookup map
■allowedStr := getEnv("ALLOWED_COUNTRY_CODES", "PK")
■countries := strings.Split(allowedStr, ",") // Returns a slice
■for _, country := range countries {
■■cleanCode := strings.TrimSpace(strings.ToUpper(country))
■■if cleanCode != "" {
■■■cfg.AllowedCountryCodes[cleanCode] = true
■■}
■}

■// 5. Unmarshal admin users array
```

```
■adminJSON := os.Getenv("ADMIN_USERS_JSON")
■if adminJSON != "" {
■■var admins []AdminUser // Allocates slice
■■if err := json.Unmarshal([]byte(adminJSON), &admins); err != nil {
■■■return nil, fmt.Errorf("decoding admin user JSON array failed: %w", err)
■■}
■■cfg.AdminUsers = admins
■}

■return cfg, nil
}

func getEnv(key, defaultVal string) string {
■val := os.Getenv(key)
■if val == "" {
■■return defaultVal
■}
■return val
}
```

Toggle Behavior to Preserve

- If `GRANT_APPLICATION_OPEN=0`, pre-registration and registration return 403.
- If `GRANT_REQUIRE_SELECTION=1`, grant submission requires whitelist selection.
- If `HFC_SHADOW_MODE=1`, HFC scores are visible but do not block approval.
- If `GEO_BLOCK_ENABLED=1`, non-allowed country codes are rejected.
- If `ACCESS_CONTROL_ENABLED=1`, applicant traffic is limited by Redis session slots.

Mastery Check

You understand this chapter when you can:

- Explain why Go handles errors as return values instead of using exceptions.
- Explain the memory differences between static arrays and slices.
- Populate and look up key presence in a map.
- Fail application startup immediately if critical configuration variables are malformed or missing.

Chapter 5: PostgreSQL, Migrations, and Data Modeling

Purpose

The database is the most important compatibility layer. The Go backend must use the same PostgreSQL schema as the Flask application. In this chapter, we will study relational modeling, Go database access, connection pools, migrations, nullable values, JSONB, and transactions.

Generics are useful in Go, but they are not the main database lesson. The main lesson is this: your Go code must treat the database as a contract, not as an afterthought.

Foundational Concepts Explained Simply

1. Relational Modeling

A relational database stores facts in tables and connects those facts with keys.

Application parallel:

- `users` stores applicant identity.
- `businesses` stores one business per applicant.
- `grants` stores one grant application per applicant.
- `business_documents` stores many uploaded documents per business.
- `grant_approval_logs` stores many approval actions per grant.

The relationships matter:

```
users 1 -> 1 businesses
users 1 -> 1 grants
businesses 1 -> many business_documents
grants 1 -> many grant_approval_logs
```

This is why blindly embedding everything in one JSON document would be a poor fit for this application. Admin reports need filtering, joining, counting, and exporting.

2. Generics in Go (Type Parameters)

Before Go 1.18, if you wanted to write a helper struct or function that could work with *any* data type, you had to use the empty interface (`interface{}` or `any`). This removed compile-time type safety, requiring you to cast values back and forth.

- **Generics** introduce **Type Parameters**, allowing you to write code that placeholder-types specify at creation time.
- **Syntax:** Place the type parameter constraint (like `[T any]` or `[T comparable]`) after the struct or function name.

Here is a simple example of a generic `Box` container:

```
package main

import "fmt"

// Box is a generic struct containing any type T
type Box[T any] struct {
    Content T
}

func main() {
    // A box holding an integer
```

```
    intBox := Box[int]{Content: 123}

    // A box holding a string
■stringBox := Box[string]{Content: "PEACE Grant"}
■
■fmt.Println(intBox.Content)    // Prints 123 (typed as int)
■fmt.Println(stringBox.Content) // Prints "PEACE Grant" (typed as string)
}
```

Application parallel: a generic paginated response can hold applicants, grants, HFC queue rows, or updates while preserving the same envelope shape.

3. Database Connection Pooling

Every SQL query requires network communication with the database engine. Establishing a raw TCP database connection is incredibly slow because it performs handshakes, SSL handshakes, and process allocations.

- **The Pool Solution:** Instead of opening a new connection for every request, a **Connection Pool** maintains a pool of persistently open, active connections.
- When Go wants to run a query, it leases a connection from the pool, executes the SQL, and releases it back to the pool.
- Go's `github.com/jackc/pgx/v5/pgxpool` manages this process concurrently and safely.

Application parallel: admin reports may be requested by multiple staff at once. Without a pool, the backend would waste time repeatedly opening connections. With a pool, requests borrow existing connections safely.

4. Nullable Values

PostgreSQL columns can be `NULL`. Go's plain `string`, `int`, and `float64` cannot represent SQL null.

Use nullable types:

```
type User struct {
    FirstName sql.NullString
    CreatedAt time.Time
}
```

When sending JSON to Vue, you may convert database rows into response DTOs so null handling is cleaner:

```
type UserProfileResponse struct {
    FirstName *string `json:"first_name"`
}
```

Application parallel: optional fields such as `middle_name`, `business_registration_number`, `approved_amount`, and `approval_reason` may be empty until later in the workflow.

5. JSONB

JSONB is PostgreSQL's binary JSON type. It is useful for flexible nested data while still keeping the main workflow relational.

Application parallel:

- `business_registration_authority` is an array.
- `financed_items` is an array of item rows.
- `employment_grid` is an object.
- `srsp_relatives` is an array of relative rows.
- `rules_triggered` is an array of HFC rules.

Beginner rule: if Go needs to validate or inspect the JSON, create typed structs. If Go only stores and returns it, `json.RawMessage` can be acceptable.

External Resources

- Go Dev: Tutorial on Generics

- pgx Connection Pool Documentation
- System Design Primer: Database Connection Pools

Tables to Preserve

The core tables are:

- `users`
- `businesses`
- `business_documents`
- `grants`
- `grant_media`
- `grant_approval_logs`
- `grant_access_whitelist`
- `applicant_status`
- `hfc_evaluations`
- `hfc_review_actions`
- `hfc_rule_config`
- `initial_registrations`
- `updates`
- `faqs`
- `schema_migrations`

All foreign keys use `ON DELETE CASCADE`. Preserve that behavior.

Application Parallel: Business Feature to Table

Portal feature	Tables touched	Go concept practiced
Login	`users`	row scanning, bcrypt hash retrieval
Business profile	`businesses`	one-to-one relation, nullable fields
Document upload	`business_documents`	one-to-many relation
Grant application	`grants`	JSONB, unique constraint
Grant approval	`grants`, `grant_approval_logs`	transaction
HFC scoring	`grants`, `hfc_evaluations`	insert plus update
Reports	many joined tables	filters, pagination, indexes

Migration Strategy

Create SQL migration files matching the original `init_db.py` history:

```
migrations/
  001_initial_users_businesses_documents.sql
  002_create_grants.sql
  ...
  023_add_grant_srsp_relatives.sql
```

Use `schema_migrations` to record applied versions.

Practical Examples

Example 1: Implementing a Generic API Response Wrapper

We can use Go's generics to build a single, type-safe HTTP response envelope structure. This ensures that whether we return a list of `User` models or a single `Business` profile, the JSON response layout remains identical:

```
// File: internal/httpx/json.go
package httpx

import (
■"encoding/json"
■"net/http"
)

// APIResponse wraps any data type T in a standard envelope structure.
type APIResponse[T any] struct {
■Success bool   `json:"success"`
■Message string `json:"message,omitempty"`
■Data    T      `json:"data,omitempty"`
}

// WriteJSON sends a generic payload with a HTTP status code.
func WriteJSON[T any](w http.ResponseWriter, status int, data T) {
■w.Header().Set("Content-Type", "application/json")
■w.WriteHeader(status)
■
■envelope := APIResponse[T]{
■■Success: status < 400,
■■Data:    data,
■}

■json.NewEncoder(w).Encode(envelope)
}
```

Example 2: Database Transaction Execution

This example demonstrates how to execute multiple writes within a single transactional block to update a grant's status and insert an approval audit log, rolling back automatically if any query fails:

```
// File: internal/grant/repository.go
package grant

import (
■"context"
■"fmt"
■"time"

■"github.com/jackc/pgx/v5"
■"github.com/jackc/pgx/v5/pgxpool"
)

type Repository struct {
■db *pgxpool.Pool
}

func NewRepository(db *pgxpool.Pool) *Repository {
■return &Repository{db: db}
}

// ApproveGrant performs a transaction: updates grant status, inserts approval audit log.
func (r *Repository) ApproveGrant(ctx context.Context, userID int64, approver string, amount
float64, reason string) error {
■// 1. Begin transaction
■tx, err := r.db.Begin(ctx)
■if err != nil {
■■return fmt.Errorf("failed to begin transaction: %w", err)
■}
■// Defer a rollback. If tx commits successfully before function ends, rollback is a safe no-
op.
■defer tx.Rollback(ctx)

■// 2. Update grant record
```

```
■updateQuery := `
■■UPDATE grants
■■SET status = 'Approved', approved_amount = $1, approval_reason = $2, approved_at = $3,
approved_by = $4
■■WHERE user_id = $5`
■
■result, err := tx.Exec(ctx, updateQuery, amount, reason, time.Now(), approver, userID)
■if err != nil {
■■return fmt.Errorf("failed to update grant: %w", err)
■}
■if result.RowsAffected() == 0 {
■■return fmt.Errorf("grant record not found for user %d", userID)
■}

■// 3. Write approval audit log
■logQuery := `
■■INSERT INTO grant_approval_logs (user_id, approved_by, approved_amount, reason, created_at)
■■VALUES ($1, $2, $3, $4, $5)`
■
■_, err = tx.Exec(ctx, logQuery, userID, approver, amount, reason, time.Now())
■if err != nil {
■■return fmt.Errorf("failed to write audit log: %w", err)
■}

■// 4. Commit transaction
■if err := tx.Commit(ctx); err != nil {
■■return fmt.Errorf("failed to commit transaction: %w", err)
■}

■return nil
}
```

Repository Pattern

Keep SQL in repository packages. Handlers should not build SQL strings directly.

Why Repositories Help Beginners

Repositories keep the code readable:

```
handler:    HTTP details
service:    business rules
repository: SQL details
```

When login fails, you know where to look:

- bad JSON body: handler
- blocked user rule: service
- wrong SQL query: repository

Project task:

```
type UserRepository interface {
    FindByEmail(ctx context.Context, email string) (*User, error)
    FindByID(ctx context.Context, userID int64) (*User, error)
}
```

Then `UserService` depends on this interface or concrete repository depending on test needs.

Mastery Check

You understand this chapter when you can:

- Explain what type parameters `[T any]` do in structures.
- Create a generic container struct.
- Explain why we use connection pools rather than spawning raw TCP queries for every request.

- Execute a multi-query transaction and explain the purpose of `defer tx.Rollback()`.

Chapter 6: Authentication, JWT, bcrypt, and Middleware

Purpose

Authentication is where backend compatibility matters immediately. The Vue frontend stores tokens and sends them as opaque bearer strings. The Go backend must issue and verify compatible JWTs. In this chapter, we will learn how Go handles functions as first-class citizens, how to use closures to configure handlers, and how `context.Context` propagates metadata. We will then build an authentication middleware chain.

Application parallel: this chapter builds the security boundary for both applicants and admins. The applicant uses `UserLogin.vue` and receives `userToken`. The admin uses `AdminLogin.vue` and receives `adminToken`. The same Go server must understand both tokens, but permissions are different.

Foundational Concepts Explained Simply

1. First-Class Functions & Closures

In Go, functions are **first-class citizens**, meaning they behave like any other value:

- You can assign a function to a variable.
- You can pass a function as an argument to another function.
- You can return a function from a function.

A **Closure** is an anonymous function that references variables from outside its immediate body. The function "closes over" those variables, carrying them along wherever it is passed.

Here is an example demonstrating a closure that returns a custom greeting handler:

```
package main

import "fmt"

// GreetingGenerator returns a function that takes a name string
func GreetingGenerator(prefix string) func(string) string {
    // This anonymous function is a closure because it references 'prefix'
    return func(name string) string {
        return fmt.Sprintf("%s, %s!", prefix, name)
    }
}

func main() {
    sayHello := GreetingGenerator("Hello")
    sayUrdu := GreetingGenerator("[redacted]")

    fmt.Println(sayHello("Aftab")) // Prints "Hello, Aftab!"
    fmt.Println(sayUrdu("Aftab"))  // Prints "[redacted], Aftab!"
}
```

2. Context (`context.Context`)

When an HTTP request enters a Go server, it creates a request-specific lifecycle.

- Go uses `context.Context` to propagate deadlines, cancellation signals, and request-scoped values down the call stack (e.g., from middleware to handlers to repositories).
- **Injecting values:** Use `context.WithValue(parentContext, key, value)`. Contexts are immutable; this returns a *new* child context containing the key-value pair.
- **Reading values:** Use `ctx.Value(key)` to fetch the value.

[!TIP] Always declare a custom, unexported type for context keys to prevent other packages from accidentally overwriting your values.

```
type contextKey string
const UserKey contextKey = "user_id"

// Injecting
ctx := context.WithValue(r.Context(), UserKey, 42)

// Extracting
userID := ctx.Value(UserKey).(int) // Type assertion
```

External Resources

- Go Tour: Function Closures
- Go Blog: Context Package Explained
- Go Web Examples: Middleware

User JWT

Payload:

```
{ "user_id": 42, "exp": 1234567890 }
```

Rules:

- HS256.
- Secret from `JWT_SECRET_KEY`.
- Expiry: 24 hours.

Beginner explanation: a JWT is a signed string. The server does not need to store it in a sessions table. The browser sends it back, and the server verifies the signature. If someone edits the `user_id` inside the token, verification fails.

Admin JWT

Payload:

```
{
  "admin_username": "admin1",
  "role": "admin",
  "is_admin": true,
  "is_approver": false,
  "exp": 1234567890
}
```

Rules:

- HS256.
- Secret from `JWT_SECRET_KEY`.
- Expiry: 8 hours.
- `is_approver` comes from `can_approve_grants`.

Application parallel: loading an admin report requires admin identity. Approving a grant requires approver identity. That is why `is_admin` and `is_approver` are separate claims.

bcrypt

Use `golang.org/x/crypto/bcrypt`:

```
err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
```

Never log plaintext passwords. Never return whether the email or password was wrong separately for normal user login.

Beginner explanation: bcrypt is intentionally slow. You never decrypt a bcrypt hash. You compare a password attempt against the stored hash. This protects users if hashes are ever exposed.

Middleware Pipeline

The original request pipeline is:

```
apply_geo_block()
apply_access_control()
authenticate_token()
```

In Go, build composable middleware:

```
handler := Recover(
    Logger(
        GeoBlock(
            AccessControl(
                Auth(router),
            ),
        ),
    ),
)
```

Admin routes skip geo-block. Public routes skip auth. Applicant-only and admin-only routes require different token claims.

Application Parallel: Route Categories

Route category	Example	Middleware needed	Identity needed
Public	`GET /api/updates`	logging, recovery	none
Public auth	`POST /api/login`	logging, recovery, applicant access controls if enabled	none
Applicant	`GET /api/business`	user auth	`user_id`
Admin	`GET /api/admin/applicants/report`	admin auth	`admin_username`, `role`
Approver	`POST /api/admin/grants/<user_id>/approve`	admin auth plus approver check	`is_approver=true`

The beginner mistake is to make one global middleware decision for every route. This portal has public pages, applicant pages, admin pages, and approver-only actions.

Context Values

After auth, attach identity to request context:

```
type Identity struct {
    UserID        int64
    AdminUsername string
    Role          string
    IsAdmin       bool
    IsApprover    bool
}
```

Handlers can read identity from context without reparsing JWTs.

Use a helper instead of type assertions everywhere:

```
func IdentityFromContext(ctx context.Context) (Identity, bool) {
    identity, ok := ctx.Value(identityKey{}).(Identity)
    return identity, ok
}
```

Application parallel: `BusinessHandler.Get` reads `user_id` from context and returns only that user's business. It should never accept `user_id` from the applicant request body for applicant-owned data.

Practical Examples

Example 1: JWT Verification Token Service

This helper generates and parses claims using `github.com/golang-jwt/jwt/v5`:

```
// File: internal/security/jwt.go
package security

import (
■"errors"
■"fmt"
■"time"

■"github.com/golang-jwt/jwt/v5"
)

type UserClaims struct {
■UserID int64 `json:"user_id"`
■jwt.RegisteredClaims
}

type JWTService struct {
■secretKey []byte
}

func NewJWTService(secret string) *JWTService {
■return &JWTService{secretKey: []byte(secret)}
}

// GenerateUserToken generates a 24-hour token for applicants.
func (s *JWTService) GenerateUserToken(userID int64) (string, error) {
■claims := UserClaims{
■■UserID: userID,
■■RegisteredClaims: jwt.RegisteredClaims{
■■■ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
■■■IssuedAt:  jwt.NewNumericDate(time.Now()),
■■},
■}

■token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
■return token.SignedString(s.secretKey)
}

// VerifyUserToken decodes and validates a user token.
func (s *JWTService) VerifyUserToken(tokenStr string) (int64, error) {
■token, err := jwt.ParseWithClaims(tokenStr, &UserClaims{}, func(t *jwt.Token) (interface{},
error) {
■■if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
■■■return nil, fmt.Errorf("unexpected signing method: %v", t.Header["alg"])
■■}
■■return s.secretKey, nil
■})

■if err != nil {
■■return 0, err
■}

■claims, ok := token.Claims.(*UserClaims)
■if !ok || !token.Valid {
■■return 0, errors.New("invalid token claims")
■}

■return claims.UserID, nil
}
```

Example 2: Closure-based Middleware injecting Context Identity

This middleware uses a closure pattern to receive dependencies (`JWTService`) and returns an `http.Handler` wrapper that injects validated identities into `r.Context()`:

```
// File: internal/security/middleware.go
package security

import (
■"context"
■"net/http"
■"strings"
)

type contextKey string

const UserIdentityKey contextKey = "userIdentity"

type Identity struct {
■UserID int64
}

// AuthMiddleware is a closure wrapping handlers with authorization checks.
func AuthMiddleware(jwtSvc *JWTService) func(http.Handler) http.Handler {
■return func(next http.Handler) http.Handler {
■■return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
■■■authHeader := r.Header.Get("Authorization")
■■■if authHeader == "" {
■■■■http.Error(w, `{"error":"missing authorization header"}`, http.StatusUnauthorized)
■■■■return
■■■}

■■■// Expecting "Bearer <token>"
■■■parts := strings.Split(authHeader, " ")
■■■if len(parts) != 2 || strings.ToLower(parts[0]) != "bearer" {
■■■■http.Error(w, `{"error":"invalid authorization format"}`, http.StatusUnauthorized)
■■■■return
■■■}

■■■userID, err := jwtSvc.VerifyUserToken(parts[1])
■■■if err != nil {
■■■■http.Error(w, `{"error":"invalid or expired token"}`, http.StatusUnauthorized)
■■■■return
■■■}

■■■// Injected Identity into Request Context
■■■ctx := context.WithValue(r.Context(), UserIdentityKey, &Identity{UserID: userID})
■■■
■■■// Call next handler in decorator chain
■■■next.ServeHTTP(w, r.WithContext(ctx))
■■})
■}
}

// GetIdentity extracts the Identity object from the context.
func GetIdentity(ctx context.Context) (*Identity, bool) {
■id, ok := ctx.Value(UserIdentityKey).(*Identity)
■return id, ok
}
```

Security Rules

Preserve:

- Blocked users cannot login.
- Applicant status does not block login.
- Admin approval endpoint requires approver identity.
- CSV export query tokens are short-lived and separate from normal bearer auth.
- Geo-block checks `CF-IPCountry` first, then `X-Country-Code`.

Mastery Check

You understand this chapter when you can:

- Explain how functions behave as values in Go.
- Write a simple closure that captures outside variables.
- Write a custom middleware function with the `func(http.Handler) http.Handler` signature.
- Propagate values securely through request scopes using `context.WithValue()`.

Chapter 7: Applicant Registration, Login, and Business Profiles

Purpose

This chapter implements the applicant foundation: closed registration behavior, login, profile lookup, and business profile create/update. In this chapter, we will study **Method Receivers**, learning the difference between attaching behaviors to structs by value vs by pointer. We will then implement struct validation receiver methods for our Request DTOs.

For a beginner Go developer, this is the first real vertical slice. You will move from HTTP JSON to service rules to SQL and back to Vue form state.

Theory: Vertical Slices

A vertical slice is a small feature implemented through every layer:

```
Vue form -> Axios request -> Go handler -> service -> repository -> PostgreSQL -> JSON
response -> Vue state
```

Building one complete slice teaches more than building ten disconnected helpers. Login and business profile are ideal first slices because they touch authentication, validation, database queries, and frontend state without requiring the full grant workflow yet.

Foundational Concepts Explained Simply

Method Receivers (Value vs. Pointer Receivers)

Go does not have classes, but you can define **Methods** on struct types. A method is simply a function with a special **Receiver** parameter listed between the `func` keyword and the method name.

- **Value Receiver:** Declared as `func (s StructType) Method()`.
- Go creates a complete **copy** of the struct when running the method.
- Modifying struct fields inside the method *does not* affect the original caller's struct.
- Use value receivers for small, read-only structures where copying is cheap.
- **Pointer Receiver:** Declared as `func (s *StructType) Method()`.
- Go passes the **memory address** of the struct instead of copying it.
- Modifying fields inside the method *directly alters* the original caller's struct.
- Use pointer receivers if the method needs to edit the struct, or if the struct is large and copying it would consume unnecessary resources.

Here is an example showing the difference:

```
package main

import "fmt"

type Circle struct {
    Radius float64
}

// Value Receiver (read-only calculation, works on copy)
func (c Circle) Area() float64 {
    return 3.14159 * c.Radius * c.Radius
}

// Pointer Receiver (modifies original, does not copy)
```

```
func (c *Circle) DoubleRadius() {
    c.Radius = c.Radius * 2
}

func main() {
    circle := Circle{Radius: 5}

    fmt.Println("Area:", circle.Area()) // Prints 78.53975

    circle.DoubleRadius()
    fmt.Println("New Radius:", circle.Radius) // Prints 10 (radius was modified)
}
```

External Resources

- A Tour of Go: Methods
- Go Dev: FAQ on Receiver Types

Endpoints

Public:

Method	Path
POST	`/api/pre-registration`
POST	`/api/register`
POST	`/api/login`
POST	`/api/forgot-password`
POST	`/api/reset-password`

User:

Method	Path
GET	`/api/user/profile`
GET	`/api/business`
POST	`/api/business`
PUT	`/api/business`

Registration Toggle

The current operational state has registration closed. Preserve:

- `/api/pre-registration` returns 403 when `GRANT_APPLICATION_OPEN=0`.
- `/api/register` returns 403 when `GRANT_APPLICATION_OPEN=0`.
- Password reset endpoints return 403 because the feature is disabled.

This is a good lesson: compatibility sometimes means reproducing disabled behavior.

Application parallel: the frontend has registration screens, but production config currently closes registration. The Go backend must enforce this even if a user manually calls the API.

Go concept: configuration-driven branching.

```
if !cfg.GrantApplicationOpen {
    return ErrRegistrationClosed
}
```

Vue concept: show the closed-registration UI when the API returns 403 or when a public status endpoint says registration is closed.

Business Profile Rules

The app allows one business per user:

```
businesses.user_id INTEGER UNIQUE NOT NULL
```

Allowed districts:

- Swat
- Shangla
- Upper Dir
- Upper Chitral
- Lower Chitral

Reject any other district in create/update logic.

Go concept: validate before write.

```
func ValidateDistrict(district string) error {
    if !AllowedDistricts[district] {
        return fmt.Errorf("district %q is not allowed", district)
    }
    return nil
}
```

Database concept: the unique constraint protects data even if two requests arrive at the same time. Application validation improves the error message, but the database is the final guard.

Request Validation

Validate at the boundary:

- Required fields.
- Email shape.
- CNIC as string.
- Boolean fields.
- JSON arrays such as `business_registration_authority`.
- District allow-list.

Do not trust the frontend. Vue validation improves UX; Go validation protects the system.

Practical Examples

Example 1: Defining a Request DTO with Pointer Receiver Validation

This example declares a data structure for business profiles and implements a validation method using a pointer receiver to optimize performance:

```
// File: internal/business/dto.go
package business

import (
■"errors"
```

```
■"strings"
)

var allowedDistricts = map[string]bool{
■"swat":          true,
■"shangla":       true,
■"upper dir":     true,
■"upper chitral": true,
■"lower chitral": true,
}

type CreateBusinessRequest struct {
■Name            string `json:"name_of_business"`
■Address         string `json:"business_full_address"`
■District        string `json:"business_location_district"`
■MaleEmployees   int    `json:"male_employees"`
■FemaleEmployees int    `json:"female_employees"`
}

// Validate uses a pointer receiver to check the request fields without copying memory.
func (req *CreateBusinessRequest) Validate() error {
■if strings.TrimSpace(req.Name) == "" {
■■return errors.New("business name is required")
■}
■if strings.TrimSpace(req.Address) == "" {
■■return errors.New("business full address is required")
■}
■
■// Check against allowed KP districts
■districtClean := strings.ToLower(strings.TrimSpace(req.District))
■if !allowedDistricts[districtClean] {
■■return errors.New("business location district is out of allowed scope")
■}

■if req.MaleEmployees < 0 || req.FemaleEmployees < 0 {
■■return errors.New("employee counts cannot be negative")
■}

■return nil
}
```

Example 2: Handler Executing DTO Validation

This controller handler decodes the request body and executes the validation method:

```
// File: internal/business/handler.go
package business

import (
■"encoding/json"
■"net/http"
■"peace-sme-go/internal/httpx"
)

type Handler struct{}

func (h *Handler) Create(w http.ResponseWriter, r *http.Request) {
■var req CreateBusinessRequest
■
■// Decode JSON directly into the struct
■if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
■■httpx.WriteError(w, http.StatusBadRequest, "invalid_json", "Failed to parse request JSON")
■■return
■}

■// Execute pointer-receiver validation
■if err := req.Validate(); err != nil {
■■httpx.WriteError(w, http.StatusUnprocessableEntity, "validation_failed", err.Error())
■■return
■}

■// Logic to pass req to Service...
}
```

Service Design

Use a service between handlers and repositories:

```
type Service struct {
    users      UserRepository
    businesses BusinessRepository
    auth       TokenService
}
```

Handlers decode HTTP. Services enforce rules. Repositories run SQL.

Beginner Walkthrough: `POST /api/business`

1. Vue gathers form fields in `reactive` state.
2. Axios sends JSON with `Authorization: Bearer <userToken>`.
3. Auth middleware verifies JWT and stores `user_id` in context.
4. Handler decodes JSON into `BusinessRequest`.
5. Service validates district and ownership.
6. Repository inserts row into `businesses`.
7. Handler returns `{message,business_id}`.
8. Vue shows success and redirects to dashboard.

Write this flow in notes before coding. It prevents mixing responsibilities.

Vue Link

These endpoints power:

- `RegistrationForm.vue`
- `UserLogin.vue`
- `AppDashboard.vue`
- `SmeBusinessProfile.vue`
- route guards using `userToken`

Keep response shapes stable so the Vue application does not need workaround code.

Mastery Check

You understand this chapter when you can:

- Explain what receiver parameters are in Go.
- Contrast value receivers and pointer receivers.
- Implement validation routines on request structures using pointer receivers.

Chapter 8: Grant Applications and Workflow Rules

Purpose

The grant workflow is the core applicant feature. It teaches larger request payloads, JSONB fields, conditional validation, idempotent updates, and workflow gates. In this chapter, we will study **Interfaces**, understanding how Go implements structural typing (duck typing) without explicit declarations to write decoupled, testable code. We will then build a state machine workflow using interfaces.

This is where the project becomes a serious Go exercise. The request is large, the rules are conditional, the database stores nested JSONB data, and the workflow interacts with HFC background scoring.

Foundational Concepts Explained Simply

1. Interfaces in Go

An **Interface** is a custom type that specifies a contract (a set of method signatures). Any struct that implements those exact methods automatically (implicitly) satisfies the interface.

- **No `implements` Keyword:** Unlike Java or C#, Go does not require you to declare that a struct implements an interface:
- If it looks like a duck and quacks like a duck, it is a duck (**Duck Typing**).
- This design decouples packages: you can define an interface locally in the package that *uses* it, rather than the package that *implements* it.
- **Polymorphism:** Allows you to swap out implementations (e.g. replacing a real database repo with a mock database repo in unit tests).

Here is a simple example:

```
package main

import "fmt"

// Greeter interface requires a Greet method
type Greeter interface {
    Greet() string
}

// English speaker struct
type English struct{}
func (e English) Greet() string {
    return "Hello"
}

// Urdu speaker struct
type Urdu struct{}
func (u Urdu) Greet() string {
    return "[redacted]"
}

// SaySomething takes any type that satisfies Greeter interface
func SaySomething(g Greeter) {
    fmt.Println(g.Greet())
}

func main() {
    SaySomething(English{}) // Prints "Hello"
    SaySomething(Urdu{})    // Prints "[redacted]"
}
```

2. State Machine Rules

A State Machine maps out a set of distinct states and defines which state changes are valid. If a user tries to jump states illegally (e.g., transitioning directly from `Draft` to `Approved` without going through the `Submitted` stage), the state machine rejects the transition.

External Resources

- A Tour of Go: Interfaces
- Effective Go: Interfaces and types

Endpoints

Method	Path	Purpose
GET	`/api/grant`	Get current user's grant plus access state
POST	`/api/grant`	Create grant application
PUT	`/api/grant`	Update existing grant
GET	`/api/grant-status`	Show status, amount, and reason

Access Rule

If `GRANT_REQUIRE_SELECTION=1`, a user must have:

```
grant_access_whitelist.is_selected = TRUE
```

before posting a grant application.

`GET /api/grant` must include an `access_state` field so the frontend can show the right UI.

Application parallel:

- Admin uses `AdminUserAccess.vue` to set whitelist status.
- Applicant dashboard reads grant access state.
- Grant form unlocks only when selected.

Go concept: gate the write in the service, not only in Vue. Vue can hide the button, but Go must reject unauthorized submission.

One Grant Per User

The schema enforces:

```
grants.user_id INTEGER UNIQUE NOT NULL
```

Use this as a design lesson:

- `POST` creates when no row exists.
- `PUT` updates an existing row.
- Duplicate creates should return a clear conflict or compatible error.

Important Fields

Grant payload includes:

- `expression_of_interest`

- `grant_required`
- `working_capital`
- `financed_items`
- `contribution_type`
- `financial_amount`
- `inkind_details`
- `grant_support_growth`
- `job_creation_details`
- `domicile_district`
- `business_type`
- `tax_registration_status`
- `employment_grid`
- `declaration_accepted`
- `has_srsp_relative`
- `srsp_relatives`

Most array/object fields map naturally to PostgreSQL JSONB.

Beginner Modeling Strategy

Do not start by writing one enormous handler. Model the request first:

```
type GrantRequest struct {
    ExpressionOfInterest []string       `json:"expression_of_interest"`
    GrantRequired        float64        `json:"grant_required"`
    FinancedItems        []FinancedItem `json:"financed_items"`
    ContributionType     string         `json:"contribution_type"`
    HasSRSPRelative      bool           `json:"has_srsp_relative"`
    SRSPRelatives        []SRSPRelative `json:"srsp_relatives"`
}
```

Then write validation as a separate function. Then write the service. Then write the handler.

This order is easier to test:

```
model -> validation -> service -> handler -> Vue
```

Validation

Implement validation in layers:

- Required grant amount.
- Valid date format.
- `financed_items` rows have item, quantity, and cost.
- If contribution includes cash, require `financial_amount`.
- If contribution includes in-kind, require details and value.
- If `has_srsp_relative=true`, require at least one relative row.
- If `declaration_accepted=false`, reject final submission.

Application parallel: every conditional Vue section must have a matching backend validation rule. If Vue shows cash fields when contribution includes cash, Go must require cash amount when contribution includes cash.

Vue condition	Go validation
contribution is cash	require `financial_amount`
contribution is in-kind	require `inkind_details` and `inkind_value`
relatives table shown	require at least one relative row
financed item added	require item name, quantity, cost
declaration checkbox	require `declaration_accepted=true`

HFC Hook

After grant create/update, enqueue HFC recalculation if `HFC_ASYNC_ENABLED=1`.

In early chapters, this can be a no-op interface:

```
type HFCEnqueuer interface {
    EnqueueRecalculate(ctx context.Context, userID int64) error
}
```

Later chapters replace it with Redis-backed work.

Beginner rule: side effects should hide behind interfaces. Grant submission should not know whether HFC is implemented as a goroutine, Redis queue, or synchronous function.

Practical Examples

Example 1: Implementing a Type-Safe State Machine

This code defines the states of a grant application and writes a validation method to enforce strict transitions:

```
// File: internal/grant/status.go
package grant

import "fmt"

type GrantStatus string

const (
■StatusDraft       GrantStatus = "Draft"
■StatusSubmitted   GrantStatus = "Submitted"
■StatusUnderReview GrantStatus = "Under Review"
■StatusApproved    GrantStatus = "Approved"
■StatusRejected    GrantStatus = "Rejected"
)

// CanTransitionTo returns an error if the state change is invalid.
func (current GrantStatus) CanTransitionTo(next GrantStatus) error {
■switch current {
■case StatusDraft:
■■if next == StatusSubmitted {
■■■return nil
■■}
■case StatusSubmitted:
■■if next == StatusUnderReview || next == StatusApproved || next == StatusRejected {
■■■return nil
■■}
■case StatusUnderReview:
■■if next == StatusApproved || next == StatusRejected {
■■■return nil
■■}
■case StatusApproved, StatusRejected:
■■return fmt.Errorf("cannot transition away from terminal state %q to %q", current, next)
■default:
■■return fmt.Errorf("unsupported source state %q", current)
■}

■return fmt.Errorf("state transition from %q to %q is disallowed", current, next)
}
```

Example 2: Interfaced Workflow Decoupling

This service layer demonstrates how interfaces allow us to process a grant submission and enqueue background scoring without coupling the code directly to specific databases or queue mechanisms:

```
// File: internal/grant/service.go
package grant

import (
■"context"
■"fmt"
)

// GrantRepository defines database behaviors needed by this service.
type GrantRepository interface {
■GetStatus(ctx context.Context, userID int64) (GrantStatus, error)
■UpdateStatus(ctx context.Context, userID int64, newStatus GrantStatus) error
}

// HFCEnqueuer defines queue behaviors needed by this service.
type HFCEnqueuer interface {
■EnqueueRecalculate(ctx context.Context, userID int64) error
}

type WorkflowService struct {
■repo   GrantRepository
■hfcSvc HFCEnqueuer
}

// NewWorkflowService injects dynamic implementations satisfying the interfaces.
func NewWorkflowService(repo GrantRepository, hfcSvc HFCEnqueuer) *WorkflowService {
■return &WorkflowService{
■■repo:   repo,
■■hfcSvc: hfcSvc,
■}
}

func (s *WorkflowService) SubmitGrant(ctx context.Context, userID int64) error {
■currentStatus, err := s.repo.GetStatus(ctx, userID)
■if err != nil {
■■return fmt.Errorf("failed to fetch grant status: %w", err)
■}

■// 1. Enforce state machine rules
■if err := currentStatus.CanTransitionTo(StatusSubmitted); err != nil {
■■return err
■}

■// 2. Perform DB write
■if err := s.repo.UpdateStatus(ctx, userID, StatusSubmitted); err != nil {
■■return fmt.Errorf("failed to write status: %w", err)
■}

■// 3. Trigger async scoring side-effect via interface
■if err := s.hfcSvc.EnqueueRecalculate(ctx, userID); err != nil {
■■fmt.Printf("Warning: failed to enqueue HFC job: %v\n", err)
■}

■return nil
}
```

Mastery Check

You understand this chapter when you can:

- Explain what implicit interface implementation means in Go.
- Create an interface and write structs that satisfy it.
- Explain how interfaces decouple packages.
- Implement state transition rules in a custom state machine.

Chapter 9: Uploads, S3-Compatible Storage, and Media References

Purpose

The portal stores documents and audio/video media in S3-compatible storage. The database stores references to public object URLs. In this chapter, we will learn about S3 object storage APIs, presigned URLs, and how to safely integrate S3 client packages in Go.

Theoretical Background

Object Storage vs. Block/File Storage

Unlike traditional filesystems which organize data into hierarchical nested directories (Block/File storage), **Object Storage** stores data as flat objects within a single bucket:

- An object consists of the file data, a unique string identifier (the **Object Key**), and metadata (such as `Content-Type`, file size, or custom tags).
- Objects are accessed via HTTP API calls (GET/PUT/DELETE requests).
- S3-compatible providers (like AWS, MinIO, Backblaze B2, Contabo) offer highly scalable flat storage structures, making them ideal for handling unstructured assets like images, PDFs, and videos.

Presigned URLs

In standard multipart uploads, a client sends the file to the backend, which in turn streams the bytes to S3 storage. This consumes substantial backend CPU, memory, and bandwidth.

- **Presigned URLs** delegate access permissions cryptographically.
- The backend uses its secret credentials to generate a temporary, signed HTTP URL (e.g., valid for 15 minutes) for a specific S3 key.
- The client uploads the file directly to S3 via a PUT request to that URL.
- This bypasses the backend for file streaming, saving significant server resources.

```
[ Client ] --- 1. Request Upload URL ---> [ Go Backend ]
[ Client ] <--- 2. Presigned PUT URL ------ [ Go Backend ]
[ Client ] --- 3. HTTP PUT File ---------> [ S3 / Object Storage ]
[ Client ] --- 4. Confirm Upload --------> [ Go Backend (Write DB reference) ]
```

Upload Validation and Security

- **MIME-Type Validation:** Never trust the client-supplied file extension. In production systems, read the first 512 bytes of the upload payload (magic bytes) to verify the MIME-Type accurately.
- **Path Traversal Prevention:** When storing files locally or naming keys on S3, clean the file names using helpers like `filepath.Base()` to prevent directory traversal exploits.
- **Authorization Verification:** Before generating a presigned URL, verify the authenticated user has write permissions for that specific business or grant directory.

External Resources

- AWS S3 Presigned URL Overview
- MinIO Go Client SDK Documentation
- OWASP File Upload Security Cheat Sheet

Endpoints

Business:

Method	Path
POST	`/api/upload-document/<business_id>`
POST	`/api/business/media/generate-upload-url`
POST	`/api/business/media/save-reference`

Grant:

Method	Path
POST	`/api/grant/generate-upload-url`
POST	`/api/grant/save-media-reference`

Applicant status:

Method	Path
POST	`/api/admin/applicant-status/generate-upload-url`

Two Upload Patterns

The app supports:

1. Multipart upload through backend.
2. Presigned URL upload directly to S3, followed by saving a DB reference.

Presigned upload is better for large media because the backend does not stream the whole file.

S3 Concepts

Learn:

- Bucket.
- Object key.
- Content type.
- ACL.
- Presigned PUT URL.
- Public base URL.
- Path-style addressing for S3-compatible providers.

Database Rows

Business documents:

```
business_documents(document_type, file_name, file_path, mime_type)
```

Grant media:

```
grant_media(media_type, file_name, file_path)
```

`file_path` is a public URL. Your Go storage package should know how to convert an object key into that URL.

Required Documents

The reports use these required document types:

- CNIC front
- CNIC back
- Business registration certificate
- Tax certificate / NTN
- Bank statement

Make document type values consistent across upload and reporting code.

Practical Examples

Example: Generating a Presigned PUT URL in Go

This complete implementation demonstrates how to configure an S3 client and generate a secure presigned URL for direct file uploads:

```
// File: internal/storage/s3.go
package storage

import (
■"context"
■"fmt"
■"time"

■"github.com/aws/aws-sdk-go-v2/aws"
■"github.com/aws/aws-sdk-go-v2/config"
■"github.com/aws/aws-sdk-go-v2/credentials"
■"github.com/aws/aws-sdk-go-v2/service/s3"
)

type S3Client struct {
■client        *s3.Client
■presignClient *s3.PresignClient
■bucketName    string
■publicBaseURL string
}

type S3Config struct {
■EndpointURL   string
■AccessKey     string
■SecretKey     string
■BucketName    string
■PublicBaseURL string
}

func NewS3Client(cfg S3Config) (*S3Client, error) {
■// Configure credentials and endpoints for S3-compatible providers
■customResolver := aws.EndpointResolverWithOptionsFunc(func(service, region string, options ...
interface{}) (aws.Endpoint, error) {
■■return aws.Endpoint{
■■■URL:           cfg.EndpointURL,
■■■SigningRegion: "us-east-1",
■■}, nil
■})

■sdkCfg, err := config.LoadDefaultConfig(context.Background(),
■■config.WithCredentialsProvider(credentials.NewStaticCredentialsProvider(cfg.AccessKey, cfg.
SecretKey, "")),
■■config.WithEndpointResolverWithOptions(customResolver),
■)
■if err != nil {
■■return nil, fmt.Errorf("failed to load S3 SDK config: %w", err)
■}
```

```
■client := s3.NewFromConfig(sdkCfg, func(o *s3.Options) {
■■o.UsePathStyle = true
■})
■
■presignClient := s3.NewPresignClient(client)

■return &S3Client{
■■client:        client,
■■presignClient: presignClient,
■■bucketName:    cfg.BucketName,
■■publicBaseURL: cfg.PublicBaseURL,
■}, nil
}

// GeneratePresignedUploadURL creates a temporary PUT URL.
func (s *S3Client) GeneratePresignedUploadURL(ctx context.Context, objectKey string,
contentType string, expiry time.Duration) (string, error) {
■input := &s3.PutObjectInput{
■■Bucket:      aws.String(s.bucketName),
■■Key:         aws.String(objectKey),
■■ContentType: aws.String(contentType),
■}

■presignedReq, err := s.presignClient.PresignPutObject(ctx, input, func(o *s3.
PresignerOptions) {
■■o.Expires = expiry
■})
■if err != nil {
■■return "", fmt.Errorf("failed to generate presigned URL: %w", err)
■}

■return presignedReq.URL, nil
}

// BuildPublicURL constructs the final public asset address.
func (s *S3Client) BuildPublicURL(objectKey string) string {
■return fmt.Sprintf("%s/%s", s.publicBaseURL, objectKey)
}
```

Security Checks

Before saving a reference:

- Verify the authenticated user owns the business.
- Verify the grant belongs to the authenticated user.
- Validate media type as `audio` or `video`.
- Validate MIME type allow-lists.
- Avoid trusting client-provided public URLs; prefer object keys and build URLs server-side.

Mastery Check

You understand this chapter when you can:

- Generate a presigned S3 PUT URL.
- Save an uploaded file reference to PostgreSQL.
- Explain why direct-to-S3 uploads reduce backend load.
- Enforce ownership before saving media.
- Build a public object URL from an object key.

Chapter 10: Redis, Caching, Access Slots, and Background Jobs

Purpose

Redis supports three separate concerns in this application: JSON response caching, concurrent applicant access control, and background job queues and HFC debounce. In this chapter, we will study **Concurrency in Go**, learning how to spawn lightweight processes (Goroutines), communicate safely between them (Channels), and coordinate operations using `select` blocks. We will then implement a background worker queue pool.

Foundational Concepts Explained Simply

1. Goroutines

A **Goroutine** is a lightweight thread of execution managed by the Go runtime.

- **Spawning:** Prefix any function call with the `go` keyword (e.g. `go doWork()`). This runs the function concurrently in the background.
- **Cost:** Spawning a goroutine is extremely cheap. They start with only ~2KB of stack space and grow dynamically, allowing you to spawn tens of thousands concurrently.

2. Channels

In Go, instead of sharing memory between threads using complex locks, goroutines communicate by sending and receiving messages over **Channels**. Channels are type-safe conduits.

- **Syntax:**
- Create: `ch := make(chan int)` (unbuffered) or `ch := make(chan int, 100)` (buffered).
- Send: `ch <- 42` (sends 42 into the channel).
- Receive: `val := <-ch` (blocks execution until a value is read from the channel).
- Close: `close(ch)` (notifies receivers that no more values will be sent).
- **Blocking:**
- Sends/receives on an **unbuffered** channel block until both the sender and receiver are ready.
- **Buffered** channels block only when the buffer is full (on sends) or empty (on receives).

3. Select Statements

The `select` statement lets a goroutine wait on multiple channel operations.

- It blocks until one of its cases is ready to execute.
- If multiple cases are ready, it chooses one pseudo-randomly.
- You can add a `default` case to make operations non-blocking.

Here is an example demonstrating goroutines, channels, and select coordination:

```
package main

import (
■"fmt"
■"time"
)

func worker(jobs <-chan int, results chan<- int) {
■for job := range jobs {
■■fmt.Printf("Processing job %d\n", job)
```

```
■■time.Sleep(100 * time.Millisecond) // Simulate slow job
■■results <- job * 2                 // Send result back
■}
}

func main() {
■jobs := make(chan int, 10)
■results := make(chan int, 10)

■// Spawn a background worker goroutine
■go worker(jobs, results)

■// Send 3 jobs
■for i := 1; i <= 3; i++ {
■■jobs <- i
■}
■close(jobs) // Close jobs to signal worker to finish loop

■// Collect results using select
■for i := 1; i <= 3; i++ {
■■select {
■■case res := <-results:
■■■fmt.Println("Result received:", res)
■■case <-time.After(500 * time.Millisecond):
■■■fmt.Println("Timeout waiting for result!")
■■}
■}
}
```

External Resources

- A Tour of Go: Goroutines
- A Tour of Go: Channels
- Go Web Examples: Channels

Cache Wrapper

Create a small package:

```
type Cache struct {
    client *redis.Client
    prefix string
}
```

Methods:

- `GetJSON(ctx, key, &dst)`
- `SetJSON(ctx, key, value, ttl)`
- `DeletePrefix(ctx, prefix)`

Use the configured prefix, defaulting to `peace_sme`.

Cached Data

Cache:

- Updates: 3600 seconds.
- FAQs: 300 seconds.
- Dashboard stats: 60 seconds.
- Report filter options: 120 seconds.

Invalidate content caches after admin creates, updates, or deletes FAQs and updates.

Access Slots

The Flask pipeline limits active applicants:

- `ACCESS_CONTROL_ENABLED=1`
- `MAX_ACTIVE_APPLICANTS=300`
- `ACCESS_SLOT_TTL_SEC=90`

In Go:

1. Determine session ID.
2. `SETEX peace_sme:session:<id> 90 1`
3. Count active session keys.
4. Return 429 if the limit is exceeded.

For production scale, avoid expensive `KEYS`; prefer `SCAN` or a sorted-set design. Preserve behavior first, optimize second.

Background Jobs

The original uses RQ queues:

- `emails`
- `hfc`

In Go, choose one of two paths:

1. Keep compatibility with existing Redis queue format while migrating gradually.
2. Build a Go worker with explicit Redis streams/lists and migrate worker behavior too.

For learning, start with an interface:

```
type JobQueue interface {
    EnqueueEmail(ctx context.Context, job EmailJob) error
    EnqueueHFC(ctx context.Context, userID int64) error
}
```

Then implement Redis behind it.

HFC Debounce

Before enqueueing HFC:

- Set a Redis debounce key for user ID.
- If key already exists, skip enqueue.
- TTL comes from `HFC_ENQUEUE_DEBOUNCE_SEC`, default 60.

Practical Examples

Example 1: Caching Helper with JSON Serialization

This helper maps database objects into Redis string keys using JSON marshaling:

```
// File: internal/cache/redis.go
package cache

import (
■"context"
■"encoding/json"
■"fmt"
■"time"

■"github.com/redis/go-redis/v9"
)

type Cache struct {
■client *redis.Client
■prefix string
}

func NewCache(client *redis.Client, prefix string) *Cache {
■return &Cache{client: client, prefix: prefix}
}

func (c *Cache) makeKey(key string) string {
■return fmt.Sprintf("%s:%s", c.prefix, key)
}

// GetJSON retrieves a cached item and unmarshals it into dst.
func (c *Cache) GetJSON(ctx context.Context, key string, dst interface{}) (bool, error) {
■fullKey := c.makeKey(key)
■val, err := c.client.Get(ctx, fullKey).Result()
■if err == redis.Nil {
■■return false, nil // Cache miss
■} else if err != nil {
■■return false, fmt.Errorf("redis read error: %w", err)
■}

■if err := json.Unmarshal([]byte(val), dst); err != nil {
■■return false, fmt.Errorf("failed to unmarshal cached data: %w", err)
■}

■return true, nil // Cache hit
}

// SetJSON marshals an item and stores it with a TTL.
func (c *Cache) SetJSON(ctx context.Context, key string, value interface{}, ttl time.Duration)
error {
■fullKey := c.makeKey(key)
■bytes, err := json.Marshal(value)
■if err != nil {
■■return fmt.Errorf("failed to marshal cache value: %w", err)
■}

■return c.client.Set(ctx, fullKey, bytes, ttl).Err()
}
```

Example 2: Goroutine Worker Pool for Processing Job Channels

This background processor pools jobs from a Go channel and executes them concurrently using separate worker goroutines:

```
// File: internal/worker/pool.go
package worker

import (
■"context"
■"log"
■"sync"
)

type EmailJob struct {
■To      string
■Subject string
■Body    string
}

type WorkerPool struct {
■jobChan    chan EmailJob
```

```
	numWorkers int
	wg         sync.WaitGroup
}

func NewWorkerPool(numWorkers int, bufferSize int) *WorkerPool {
	return &WorkerPool{
		jobChan:    make(chan EmailJob, bufferSize),
		numWorkers: numWorkers,
	}
}

// Start spawns the worker goroutines.
func (p *WorkerPool) Start(ctx context.Context) {
	for i := 1; i <= p.numWorkers; i++ {
		p.wg.Add(1)
		go p.worker(ctx, i) // Spawn concurrently
	}
}

// worker loops reading from the shared job channel.
func (p *WorkerPool) worker(ctx context.Context, workerID int) {
	defer p.wg.Done()
	log.Printf("Background worker %d started.", workerID)

	for {
		select {
		case <-ctx.Done():
			log.Printf("Worker %d received shutdown signal.", workerID)
			return
		case job, ok := <-p.jobChan:
			if !ok {
				log.Printf("Worker %d channel closed. Exiting.", workerID)
				return
			}

			// Process job
			p.executeJob(job, workerID)
		}
	}
}

func (p *WorkerPool) executeJob(job EmailJob, workerID int) {
	log.Printf("[Worker %d] Processing email to %s, Subject: %q", workerID, job.To, job.Subject)
	// actual integration sending goes here...
}

// Enqueue puts a job into the buffered channel.
func (p *WorkerPool) Enqueue(job EmailJob) {
	p.jobChan <- job
}

// Stop closes the channel and waits for workers to drain it.
func (p *WorkerPool) Stop() {
	close(p.jobChan)
	p.wg.Wait()
	log.Println("Worker pool stopped cleanly.")
}
```

Mastery Check

You understand this chapter when you can:

- Explain what goroutines are and why they are lightweight.
- Create unbuffered and buffered channels.
- Send and receive values on channels without locking threads.
- Coordinate multiple channels using `select` statements.
- Write a clean worker loop that drains channels safely.

Chapter 11: Admin, Reports, CSV, and Database Browser

Purpose

Admin features teach pagination, filtering, sorting, authorization, CSV streaming, and defensive SQL.

Theoretical Background

SQL Injection Prevention

SQL Injection occurs when untrusted user input is concatenated directly into SQL query strings, allowing attackers to manipulate queries.

- **Parametric Queries:** Always use parameters (`$1`, `$2` in PostgreSQL) rather than concatenation. The driver sends values to the database engine separately from the SQL command structure, neutralizing threats.
- **Dynamic Sort Columns:** You cannot use query parameters for SQL keywords or schema object names (like columns or tables in `ORDER BY`). To sort dynamically, use a strict **allow-list** mapping user-provided sorting keys to absolute SQL columns.

Pagination Strategies

1. **Offset-Based Pagination (`LIMIT X OFFSET Y`):** Simple to implement and allows direct page jumping.
   - **Performance issue:** As `OFFSET` grows, the database must scan and discard all records leading up to the offset, degrading performance on millions of rows.
1. **Cursor-Based (Keyset) Pagination (`WHERE id > last_seen_id LIMIT X`):** Highly performant and works on a constant execution plan.
   - **Trade-off:** Does not allow jumping directly to page 50 without loading pages 1-49 first.

Memory-Efficient CSV Streaming

When exporting millions of database rows to a CSV file:

- **Do not** fetch all records into an in-memory array first, as this causes RAM exhaustion.
- Instead, query the database, retrieve a row iterator, and stream rows line-by-line directly to the HTTP response writer. Using `encoding/csv` flushed periodically ensures constant memory usage.

External Resources

- Use The Index, Luke! - Keyset Pagination Guide
- OWASP SQL Injection Prevention Cheat Sheet
- Go encoding/csv package documentation

Admin Areas

Implement:

- Applicants list.
- Applicant detail dossier.
- Applicant reports.

- Submitted grants report.
- Approved grants report.
- Missing documents report.
- Fully verified report.
- Eligibility criteria report.
- Dashboard stats and frequency charts.
- Updates and FAQ management.
- User blocking and password reset stubs.
- Database browser.

Pagination Shape

Preserve:

```
{
  "data": [],
  "total": 0,
  "page": 1,
  "per_page": 20
}
```

The frontend depends on this shape.

Filtering

Reports filter by:

- document status
- language
- gender
- search
- district
- sector
- status
- date range
- HFC status
- amount range

Build filters with parameterized SQL. Never concatenate user input directly into SQL values.

For `sort_by`, use an allow-list:

```
var allowedSorts = map[string]string{
    "created_at": "u.created_at",
    "district": "b.business_location_district",
}
```

CSV Exports

CSV export endpoints use a short-lived query token:

- `/api/admin/reports/full-applicant-profiles/csv?token=<query_token>`
- `/api/admin/reports/business-profiles/csv?token=<query_token>`

Do not expose CSV exports with no auth. Query-token auth exists because browsers have trouble attaching bearer headers to direct file downloads.

Database Browser

Endpoints:

- `GET /api/admin/db-browser/tables`
- `GET /api/admin/db-browser/data/<table_name>`

Security rule:

- Table names must come from `information_schema.tables`.
- Do not let arbitrary path input become raw SQL.

Use an allow-list built from the database table list.

Content Management

Updates and FAQs are simple CRUD, but they teach cache invalidation:

- Public `/api/updates` and `/api/faqs` can be cached.
- Admin writes must invalidate those cache keys.
- FAQ search can be uncached or cached by query if needed.

Practical Examples

Example 1: Defensive Dynamic SQL Builder for Paginated Reports

This example constructs a parameterized query with dynamic filters and validates sorting parameters defensively against SQL injection:

```
// File: internal/report/repository.go
package report

import (
■"context"
■"fmt"
■"strings"

■"github.com/jackc/pgx/v5/pgxpool"
)

type ReportFilter struct {
■District string
■Status   string
■Page     int
■PerPage  int
■SortBy   string
}

// Allow-list for Order By clauses
var allowedSortColumns = map[string]string{
■"id":         "g.grant_id",
■"amount":     "g.grant_required",
■"created_at": "g.application_date",
}
```

```
func (r *Repository) GetGrantsReport(ctx context.Context, f ReportFilter) ([]GrantSummary,
int64, error) {
■var conditions []string
■var args []interface{}
■argCounter := 1

■// Add dynamic status filter safely
■if f.Status != "" {
■■conditions = append(conditions, fmt.Sprintf("g.status = $%d", argCounter))
■■args = append(args, f.Status)
■■argCounter++
■}

■// Add dynamic district filter safely
■if f.District != "" {
■■conditions = append(conditions, fmt.Sprintf("g.domicile_district = $%d", argCounter))
■■args = append(args, f.District)
■■argCounter++
■}

■// Build WHERE clause
■whereClause := ""
■if len(conditions) > 0 {
■■whereClause = "WHERE " + strings.Join(conditions, " AND ")
■}

■// Validate sorting inputs defensively
■sortCol, ok := allowedSortColumns[f.SortBy]
■if !ok {
■■sortCol = "g.application_date" // fallback default
■}

■// Run total count query
■countQuery := fmt.Sprintf("SELECT COUNT(*) FROM grants g %s", whereClause)
■var total int64
■err := r.db.QueryRow(ctx, countQuery, args...).Scan(&total)
■if err != nil {
■■return nil, 0, fmt.Errorf("failed to count records: %w", err)
■}

■// Build main query with Pagination limiters
■limit := f.PerPage
■offset := (f.Page - 1) * f.PerPage

■mainQuery := fmt.Sprintf(`
■■SELECT g.grant_id, g.user_id, g.status, g.grant_required
■■FROM grants g
■■%s
■■ORDER BY %s DESC
■■LIMIT $%d OFFSET $%d`, whereClause, sortCol, argCounter, argCounter+1)

■args = append(args, limit, offset)

■// Execute mainQuery and scan rows...
■return summaries, total, nil
}
```

Example 2: Streaming CSV Direct to Client

This example streams database records as CSV directly into an HTTP response using constant memory overhead:

```
// File: internal/report/handler.go
package report

import (
■"encoding/csv"
■"log"
■"net/http"
■"strconv"
)

type Handler struct {
■db *pgxpool.Pool
}

func (h *Handler) StreamCSVExport(w http.ResponseWriter, r *http.Request) {
■// Set correct response headers for direct download
```

```
    w.Header().Set("Content-Type", "text/csv")
    w.Header().Set("Content-Disposition", "attachment;filename=grants_export.csv")

    // Instantiate standard Go CSV writer writing directly to the ResponseWriter
    writer := csv.NewWriter(w)
    defer writer.Flush()

    // Write CSV headers
    if err := writer.Write([]string{"Grant ID", "User ID", "Amount Requested"}); err != nil {
        log.Printf("Failed to write CSV header: %v", err)
        return
    }

    // Open connection and iterate rows row-by-row
    rows, err := h.db.Query(r.Context(), "SELECT grant_id, user_id, grant_required FROM grants")
    if err != nil {
        http.Error(w, "database read failure", http.StatusInternalServerError)
        return
    }
    defer rows.Close()

    for rows.Next() {
        var grantID, userID int64
        var amount float64
        if err := rows.Scan(&grantID, &userID, &amount); err != nil {
            log.Printf("Failed to scan row: %v", err)
            return
        }

        record := []string{
            strconv.FormatInt(grantID, 10),
            strconv.FormatInt(userID, 10),
            strconv.FormatFloat(amount, 'f', 2, 64),
        }

        if err := writer.Write(record); err != nil {
            log.Printf("Failed to write CSV row: %v", err)
            return
        }

        // Flush buffer to client periodically
        writer.Flush()
    }
}
```

Mastery Check

You understand this chapter when you can:

- Build paginated SQL with total count.
- Add filters without SQL injection.
- Stream a CSV response.
- Protect admin-only routes.
- Invalidate cache after content writes.
- Build a database browser without allowing arbitrary SQL execution.

Chapter 12: HFC Fraud Detection and Approval Authority

Purpose

HFC teaches deterministic scoring, background recalculation, audit trails, and role-based approvals.

Theoretical Background

Rule-Based Engines vs. ML Models

In fraud detection and risk scoring, systems use one of two main approaches:

1. **Deterministic Rule-Based Engines:** Apply strict, logical heuristics to evaluate input data (e.g., "If email already exists, add 20 points").

- **Pros:** 100% transparent, predictable, easy to debug, and requires no training datasets.
- **Cons:** Rigid; cannot easily detect novel fraud patterns that do not violate explicit rules.

1. **Machine Learning Classifiers:** Make probabilistic predictions based on feature sets learned from historical data.

- **Pros:** Can detect complex, non-linear fraud signals.
- **Cons:** Hard to explain (black-box problem), prone to data drift, and slower to execute.

This application utilizes a deterministic rule engine (v1) as its primary scorer, with optional hook points for ML scoring.

Message Processing Guarantees

When background workers process jobs (such as recalculating HFC scores or sending approval notifications), three execution guarantees are possible:

- **At-Most-Once:** The job is pulled from the queue and immediately marked complete before execution. If the worker crashes mid-job, the task is lost.
- **At-Least-Once (Recommended):** The job remains in the queue (or in a processing state) until the worker explicitly acknowledges completion. If the worker crashes, the job is re-run. This requires writing **Idempotent** tasks (tasks that can run multiple times safely without duplicate side-effects).
- **Exactly-Once:** Achieved by combining At-Least-Once processing with deduplication mechanisms (like database unique indexes or transaction locks).

Transactional Outbox Pattern

When performing actions that involve writing to a database and triggering an external side effect (like sending an email or publishing to a message broker):

- **Do not** call the external service inside the database transaction. If the service is slow, it holds the DB connection open. If the transaction rolls back, the email cannot be unsent (dual-write problem).
- **Solution:** Use the Transactional Outbox pattern. Write both the primary state change (e.g., `ApproveGrant`) and a task record (e.g., `SendEmailJob`) to the database inside the same transaction. A separate background worker reads the outbox table and executes the side-effects.

External Resources

- Distributed Systems - Messaging Guarantees

- Microservices Pattern: Transactional Outbox
- Wikipedia: Rule-Based System

HFC Rules

Score rules:

Rule	Points
Duplicate CNIC	50
Duplicate email	20
Duplicate mobile	20
Missing business profile	30
Missing required documents	25
Missing grant media	10
Business district out of scope	40
Grant amount above threshold	15
Application submitted very fast	10
Missing expression of interest	15

Risk bands:

Score	Level
0-29	LOW
30-59	MEDIUM
60-79	HIGH
80+	CRITICAL

Tables

HFC writes:

- `hfc_evaluations`
- `hfc_review_actions`
- HFC fields on `grants`

Approval writes:

- `grants.status`
- `grants.approved_amount`
- `grants.approval_reason`
- `grants.approved_at`
- `grants.approved_by`
- `grant_approval_logs`

Admin Endpoints

HFC:

- `GET /api/admin/hfc/dashboard/stats`
- `GET /api/admin/hfc/queue`
- `GET /api/admin/hfc/applicant/<user_id>`
- `POST /api/admin/hfc/applicant/<user_id>/action`
- `POST /api/admin/hfc/applicant/<user_id>/recalculate`

Approval:

- `GET /api/admin/grants/<user_id>/approval-check`
- `POST /api/admin/grants/<user_id>/approve`

Shadow Mode

If `HFC_SHADOW_MODE=1`, HFC score does not block approval. The score is advisory.

If shadow mode is off, define explicit blocking behavior before implementation. For example, require override for HIGH/CRITICAL risk.

Audit Trails

Every admin action should record:

- actor username
- action type
- before state
- after state
- comment
- timestamp

Audit records are not decoration. They are how an approval system explains itself later.

Practical Examples

Example: Implementing a Deterministic HFC Scorer in Go

This example shows how to write a scoring engine that queries various database tables and calculates a cumulative fraud score based on the application rules:

```
// File: internal/hfc/scorer.go
package hfc

import (
■"context"
■"fmt"
)

type EvaluationResult struct {
■Score     int
■RiskLevel string
■Details   map[string]int
}

type ScorerRepository interface {
■CheckDuplicateCNIC(ctx context.Context, userID int64, cnic string) (bool, error)
```

```
■HasBusinessProfile(ctx context.Context, userID int64) (bool, error)
■GetUploadedDocumentsCount(ctx context.Context, userID int64) (int, error)
}

type Scorer struct {
■repo ScorerRepository
}

func NewScorer(repo ScorerRepository) *Scorer {
■return &Scorer{repo: repo}
}

// Evaluate calculates the risk score and risk bands for an applicant.
func (s *Scorer) Evaluate(ctx context.Context, userID int64, cnic string) (*EvaluationResult,
error) {
■score := 0
■details := make(map[string]int)

■// Rule 1: Check for duplicate CNIC
■isDuplicateCNIC, err := s.repo.CheckDuplicateCNIC(ctx, userID, cnic)
■if err != nil {
■■return nil, fmt.Errorf("failed to check duplicate CNIC: %w", err)
■}
■if isDuplicateCNIC {
■■score += 50
■■details["duplicate_cnic"] = 50
■}

■// Rule 2: Check for missing business profile
■hasProfile, err := s.repo.HasBusinessProfile(ctx, userID)
■if err != nil {
■■return nil, fmt.Errorf("failed to check business profile: %w", err)
■}
■if !hasProfile {
■■score += 30
■■details["missing_business_profile"] = 30
■}

■// Rule 3: Check for incomplete documents (expecting 5 required uploads)
■uploadedCount, err := s.repo.GetUploadedDocumentsCount(ctx, userID)
■if err != nil {
■■return nil, fmt.Errorf("failed to get docs count: %w", err)
■}
■if uploadedCount < 5 {
■■score += 25
■■details["missing_required_documents"] = 25
■}

■// Determine Risk Level Band
■var riskLevel string
■switch {
■case score <= 29:
■■riskLevel = "LOW"
■case score <= 59:
■■riskLevel = "MEDIUM"
■case score <= 79:
■■riskLevel = "HIGH"
■default:
■■riskLevel = "CRITICAL"
■}

■return &EvaluationResult{
■■Score:     score,
■■RiskLevel: riskLevel,
■■Details:   details,
■}, nil
}
```

Mastery Check

You understand this chapter when you can:

- Implement deterministic scoring from database facts.

- Store a full HFC evaluation record.
- Update grant HFC fields from latest evaluation.
- Restrict approval to `is_approver`.
- Record approval logs.
- Explain why shadow mode changes enforcement but not scoring.

Chapter 13: Vue 3 Frontend Foundations

Purpose

The Vue frontend teaches SPA structure, routing, state stored in localStorage, Axios API calls, bilingual rendering, and component composition.

Application parallel: every Vue concept in this chapter is tied to a PEACE SME screen. You learn `ref` by storing login fields, `reactive` by building the business profile form, `computed` by switching English/Urdu labels, and route guards by protecting applicant/admin pages.

Theoretical Background

SPA vs. MPA Architecture

Traditional web applications (Multi-Page Applications or **MPAs**) fetch a completely new HTML document from the server on every navigation link clicked:

- **Single Page Applications (SPAs)** load a single HTML shell page once.
- Navigation is handled client-side using JavaScript (via the **HTML5 History API** or hash changes).
- The router dynamically updates the DOM and swaps views without performing full page refreshes, resulting in smooth transitions.
- Data fetching occurs in the background via asynchronous HTTP requests (`fetch` or `Axios`), swapping JSON payloads.

Vue 3 Reactivity System (ES6 Proxies)

Vue 3 uses JavaScript **ES6 Proxies** to drive its reactivity engine:

- When a reactive object is initialized (`reactive()` or `ref()`), Vue wraps it in a Proxy.
- **Dependency Tracking (Get):** When a component renders and accesses a reactive property, Vue records it as a dependency (called "tracking").
- **Visual Re-rendering (Set):** When a reactive property changes, the proxy traps the write, notifying the dependent components to re-run their render functions and update the DOM (called "triggering").

```
[ Data Change (Set) ] -> [ Proxy Trap ] -> [ Trigger Effect ] -> [ Virtual DOM Diff ] -> [ DOM Render ]
```

Composition API vs. Options API

- **Options API:** Groups component code by option types: `data()`, `methods`, `computed`, `watch`. This can lead to fragmented feature logic in large components.
- **Composition API (Vue 3):** Uses a single `setup()` function or `<script setup>` syntax. It allows you to group code by logical features and extract stateful logic into reusable functions called **Composables** (e.g., `useAuth()`).

External Resources

- Vue.js Official Guide: Reactivity in Depth
- Vue Router Documentation
- Axios Getting Started Guide

Core Vue Concepts

Learn:

- `createApp`
- Single File Components
- Composition API
- `ref`
- `reactive`
- `computed`
- `watch`
- `onMounted`
- props and emits
- Vue Router
- Axios interceptors
- Tailwind utility classes

Concept to Portal Feature

Vue concept	PEACE SME use
`ref`	login email, password, loading, error
`reactive`	business profile form and grant form
`computed`	selected translation object and RTL state
`watch`	reload admin reports when filters change
`onMounted`	fetch dashboard profile and announcements
props	reusable input, upload, and table components
emits	upload complete, modal close, row selected
router guard	redirect unauthenticated users
Axios interceptor	attach JWT bearer token

Application Entry

Recommended structure:

```
frontend/src/
  main.js
  App.vue
  apiConfig.js
  router/index.js
  api/client.js
  views/
  components/
```

`apiConfig.js` is the single source of truth for the backend base URL.

Route Guards

Preserve:

```
requiresAuth  -> localStorage.userToken  -> else /login
requiresAdmin -> localStorage.adminToken -> else /admin/login
```

Do not rename token keys unless you update all dependent code.

Application parallel:

- `/dashboard` requires `userToken`.
- `/business-profile` requires `userToken`.
- `/grant-application` requires `userToken`.
- `/admin/dashboard` requires `adminToken`.
- `/admin/grants/submitted/:id` requires `adminToken`.

Frontend guards improve navigation, but the Go backend remains the real security boundary.

Axios Client

Create one Axios instance. Attach tokens based on route area or helper calls:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('userToken')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

For admin calls, either use a second admin client or explicit helper that attaches `adminToken`.

Bilingual UI

Language storage:

```
localStorage.setItem('language', 'urdu')
```

Computed state:

```
const isUrdu = computed(() => localStorage.getItem('language') === 'urdu')
```

Rendering:

```
<h1 :class="{ 'text-right font-urdu': isUrdu }">{{ t.title }}</h1>
```

Store values in English even when labels render in Urdu.

Tailwind Discipline

Use Tailwind for layout, spacing, typography, and responsive states. Keep repeated visual patterns in components:

- `Navbar.vue`
- `AdminNavbar.vue`
- `Footer.vue`
- drawers
- modals
- upload controls
- table controls

Practical Examples

Example 1: Vue 3 Single File Component (SFC)

This complete example demonstrates the Composition API (`<script setup>`) syntax, reactivity (`ref`, `computed`), lifecycle hooks (`onMounted`), and dynamic RTL layouts for Urdu translation:

```
<!-- File: src/views/WelcomeBanner.vue -->
<script setup>
import { ref, computed, onMounted } from 'vue'

// Reactive state
const name = ref('')
const language = ref(localStorage.getItem('language') || 'english')

// Translation object
const translations = {
  english: {
    welcome: 'Welcome to PEACE SME Grant Portal',
    placeholder: 'Enter your name',
  },
  urdu: {
    welcome: '[illegible]',
    placeholder: '[illegible]',
  }
}

// Computed property for language detection
const isUrdu = computed(() => language.value === 'urdu')

// Computed translations
const t = computed(() => translations[language.value])

// Lifecycle Hook
onMounted(() => {
  console.log('Welcome component mounted successfully.')
})
</script>

<template>
  <div :class="[isUrdu ? 'text-right rtl font-urdu' : 'text-left ltr', 'p-6 bg-white shadow
rounded-lg']">
    <h1 class="text-2xl font-bold text-slate-800 mb-4">
      {{ t.welcome }} <span v-if="name" class="text-blue-600">{{ name }}</span>
    </h1>

    <label class="block text-sm font-medium text-gray-700 mb-2">{{ t.placeholder }}</label>
    <input
      v-model="name"
      type="text"
      class="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md focus:outline-none
focus:ring-blue-500 focus:border-blue-500"
      :placeholder="t.placeholder"
    />
  </div>
</template>

<style scoped>
.font-urdu {
  font-family: 'Noto Nastaliq Urdu', sans-serif;
  line-height: 2.2;
}
</style>
```

Example 2: Configuring Axios Interceptors globally

This example shows how to set up an Axios instance that intercepts all requests to inject bearer tokens from `localStorage` dynamically:

```
// File: src/api/client.js
import axios from 'axios'
import apiConfig from '../apiConfig'

const apiClient = axios.create({
  baseURL: apiConfig.baseURL,
  timeout: 10000, // 10 seconds
  headers: {
    'Content-Type': 'application/json'
  }
})
```

```
// Request Interceptor: Attach authentication tokens dynamically
apiClient.interceptors.request.use(
  (config) => {
    // Check if route is admin, use admin token; else use user token
    const isAdminRoute = window.location.pathname.startsWith('/admin')
    const tokenKey = isAdminRoute ? 'adminToken' : 'userToken'
    const token = localStorage.getItem(tokenKey)

    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }

    return config
  },
  (error) => {
    return Promise.reject(error)
  }
)

// Response Interceptor: Handle errors (like 401 Unauthorized) globally
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      log.error('Unauthorized access detected. Redirecting to login.')
      // Invalidate tokens and redirect
      localStorage.removeItem('userToken')
      localStorage.removeItem('adminToken')
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)

export default apiClient
```

Mastery Check

You understand this chapter when you can:

- Build a route-protected Vue page.
- Fetch backend JSON with Axios.
- Store and read JWTs from localStorage.
- Switch English/Urdu labels without changing stored values.
- Use Composition API instead of Options API for new code.

Chapter 14: Vue Applicant and Admin Interfaces

Purpose

This chapter turns Vue fundamentals into full application screens.

Theoretical Background

Form State Management (Wizard Pattern)

Large multi-step forms (like the 9-section Grant Application form) require strict state management:

- **Single Source of Truth:** Keep form data in a single, root-level reactive object (`reactive()`). This avoids syncing child component state manually.
- **Wizard Flow Patterns:** Break the layout into steps. A reactive `currentStep` index determines which section renders (`v-if="currentStep === X"`).
- **Validation Gates:** Before allowing a user to progress to the next step, execute validation logic on the subfields of the current step.
- **Dirty Checking:** Track if the user has modified fields without saving, allowing you to prompt a warning before they close the tab or navigate away.

Chart Rendering and Component Cleanup

When rendering visualizations (like frequency reports or risk distributions) using Canvas-based libraries (Chart.js):

- Canvas rendering requires a physical DOM node to exist on the page. Therefore, chart initialization must take place inside the `onMounted()` hook.
- **Preventing Memory Leaks:** If a user navigates to another page, the component is unmounted but the chart instance may still exist in heap memory. Always store a reference to the chart instance and destroy it in the `onBeforeUnmount()` lifecycle hook.

External Resources

- Vue.js State Management Guide
- Chart.js Integration with Vue

Public Views

Build:

- Landing page with updates and FAQ access.
- Language selection.
- Terms and conditions.
- Registration form.
- User login.
- Admin login.
- Waiting room.
- Geo-blocked page.
- Password reset disabled pages.

Applicant Views

Build:

- Dashboard.
- Business profile form.
- Grant application form.
- Grant review page.
- Upload flows for documents and media.

The grant form is large. Split it into sections:

1. Applicant identification.
2. Business details.
3. Purpose of grant.
4. Items to be financed.
5. Contribution.
6. Business growth.
7. Disclaimer.
8. Declaration.
9. How did you hear.

Use child components when a section has independent state and validation.

Admin Views

Build:

- Admin dashboard.
- Submitted grants table.
- Grant detail.
- Applicant status.
- Grant access whitelist.
- Applicant report.
- Applied details report.
- Approved grants report.
- Missing documents report.
- Eligibility criteria report.
- Full report.
- HFC dashboard, queue, applicant detail, model tuning.
- Updates management.
- FAQ management.
- Database browser.

Form State Pattern

Use:

```
const form = reactive({
  grant_required: null,
  financed_items: [],
  declaration_accepted: false
})
```

For repeating rows:

```
function addItem() {
  form.financed_items.push({ item: '', quantity: 1, estimated_cost: null })
}
```

Admin Tables

Tables need:

- loading state
- error state
- empty state
- pagination controls
- sortable headers
- filters
- CSV export action where supported

Keep query state in the URL when useful so admins can refresh or share filtered views.

Charts

Use Chart.js for:

- dashboard counts
- registration/application frequency
- HFC status distribution
- eligibility criteria breakdown

Charts should be backed by API responses, not hardcoded data.

Practical Examples

Example: Multi-step Wizard Form with Validation and State

This example demonstrates a complete Vue component layout implementing a multi-step form wizard with inline step validation and repeating row creation:

```
<!-- File: src/components/GrantWizard.vue -->
<script setup>
import { ref, reactive, computed } from 'vue'

const currentStep = ref(1)

// Root Single Source of Truth form state
const form = reactive({
```

```
  applicantName: '',
  financedItems: [
    { name: '', qty: 1, cost: 0 }
  ]
})

// Validation per step
const stepErrors = ref([])

const isStepValid = () => {
  stepErrors.value = []

  if (currentStep.value === 1) {
    if (!form.applicantName.trim()) {
      stepErrors.value.push('Applicant name is required.')
    }
  }

  if (currentStep.value === 2) {
    if (form.financedItems.length === 0) {
      stepErrors.value.push('At least one item must be added.')
    }
    for (let i = 0; i < form.financedItems.length; i++) {
      if (!form.financedItems[i].name.trim() || form.financedItems[i].cost <= 0) {
        stepErrors.value.push(`Item ${i + 1} has invalid name or estimated cost.`)
      }
    }
  }

  return stepErrors.value.length === 0
}

const nextStep = () => {
  if (isStepValid()) {
    currentStep.value++
  }
}

const prevStep = () => {
  currentStep.value--
}

// Add/Remove repeating item rows
const addRow = () => {
  form.financedItems.push({ name: '', qty: 1, cost: 0 })
}

const removeRow = (index) => {
  form.financedItems.splice(index, 1)
}

// Compute total requested cost
const totalCost = computed(() => {
  return form.financedItems.reduce((sum, item) => sum + (item.qty * item.cost), 0)
})

const submitForm = () => {
  if (isStepValid()) {
    console.log('Submitting payload to /api/grant:', form)
    // api.post('/api/grant', form) ...
  }
}
</script>

<template>
  <div class="max-w-xl mx-auto p-8 bg-white border border-slate-200 rounded-xl shadow-lg">
    <!-- Step Indicators -->
    <div class="flex items-center justify-between mb-8">
      <span :class="['font-bold', currentStep >= 1 ? 'text-blue-600' : 'text-slate-400']">1.
Identity</span>
      <span class="w-12 h-0.5 bg-slate-300"></span>
      <span :class="['font-bold', currentStep >= 2 ? 'text-blue-600' : 'text-slate-400']">2.
Financed Items</span>
    </div>

    <!-- Validation Warnings -->
    <div v-if="stepErrors.length > 0" class="mb-6 p-4 bg-red-50 border-l-4 border-red-500 text-
```

```
red-700">
      <ul>
        <li v-for="err in stepErrors" :key="err" class="text-sm">{{ err }}</li>
      </ul>
    </div>

    <!-- Step 1 View -->
    <div v-if="currentStep === 1" class="space-y-4">
      <label class="block text-sm font-semibold text-slate-700">Full Name of Applicant</label>
      <input
        v-model="form.applicantName"
        type="text"
        class="w-full px-4 py-2 border border-slate-300 rounded-lg focus:ring-2 focus:ring-
blue-500"
      />
    </div>

    <!-- Step 2 View -->
    <div v-if="currentStep === 2" class="space-y-4">
      <h3 class="text-lg font-bold text-slate-800">Items to be Financed</h3>

      <div v-for="(item, idx) in form.financedItems" :key="idx" class="flex gap-4 items-
center">
        <input
          v-model="item.name"
          placeholder="Item name"
          class="flex-1 px-3 py-2 border rounded-md"
        />
        <input
          v-model.number="item.qty"
          type="number"
          placeholder="Qty"
          class="w-16 px-3 py-2 border rounded-md"
        />
        <input
          v-model.number="item.cost"
          type="number"
          placeholder="Cost"
          class="w-24 px-3 py-2 border rounded-md"
        />
        <button
          @click="removeRow(idx)"
          class="text-red-500 font-bold hover:text-red-700"
        >
          &times;
        </button>
      </div>

      <button
        @click="addRow"
        class="text-sm text-blue-600 hover:underline"
      >
        + Add Another Item
      </button>

      <div class="pt-4 border-t text-right font-bold text-slate-800">
        Total Estimated Cost: ${{ totalCost }}
      </div>
    </div>

    <!-- Navigation Buttons -->
    <div class="mt-8 flex justify-between">
      <button
        v-if="currentStep > 1"
        @click="prevStep"
        class="px-4 py-2 bg-slate-100 hover:bg-slate-200 rounded-lg"
      >
        Back
      </button>
      <div class="ml-auto">
        <button
          v-if="currentStep < 2"
          @click="nextStep"
          class="px-5 py-2 bg-blue-600 hover:bg-blue-700 text-white rounded-lg"
        >
          Next
        </button>
```

```
        <button
          v-else
          @click="submitForm"
          class="px-5 py-2 bg-emerald-600 hover:bg-emerald-700 text-white rounded-lg"
        >
          Submit Application
        </button>
      </div>
    </div>
  </div>
</template>
```

Mastery Check

You understand this chapter when you can:

- Build a large form without losing state.
- Create reusable admin table controls.
- Keep Vue labels bilingual.
- Connect every screen to the matching `/api` endpoint.
- Handle loading, error, empty, and success states professionally.

Chapter 15: Testing, Debugging, and API Compatibility

Purpose

Testing proves that the Go and Vue rewrite matches the Flask behavior. Compatibility is the product.

Theoretical Background

The Testing Pyramid

A balanced software testing strategy is represented as a pyramid:

1. **Unit Tests (Base):** Focus on isolated functions (e.g., config loaders, score calculators). They are fast, reliable, and run entirely in memory.
2. **Integration Tests (Middle):** Validate that multiple components (e.g., service layers interacting with a real PostgreSQL connection pool or S3 mock) work together correctly.
3. **End-to-End (E2E) Tests (Top):** Spin up the entire system (Vue frontend + Go backend + DB + Redis) and simulate real user flows via browser automated drivers (like Cypress or Playwright).

```
      /\
     /  \     <-- E2E Tests (Slow, High Fidelity)
    /----\
   /      \   <-- Integration Tests (Medium Speed)
  /--------\
 /          \ <-- Unit Tests (Fast, In-Memory)
/------------\
```

Table-Driven Testing in Go

Go's idiomatic pattern for unit testing is **Table-Driven Testing**:

- You define a slice of anonymous structs representing input parameters, expected outputs, and test description names.
- You iterate over this "table" of test cases, running each case as an isolated subtest using `t.Run()`.
- This keeps test files clean, makes it easy to add new test vectors, and avoids repetitive copy-paste testing code.

Keyset Regression & API Compatibility

When migrating a backend:

- The HTTP path, request verbs, response JSON keys, and HTTP status codes must match the original implementation exactly.
- Test suites must verify contract details: e.g., verifying that a missing field returns exactly `422 Unprocessable Entity` containing the specific error keys expected by Axios interceptors.

Git Bisect for Debugging Regressions

When a regression bug is introduced in a codebase and it is unclear which commit caused it:

- **Git Bisect** executes a binary search through your commit history.
- You mark a known "good" commit (where the bug did not exist) and a known "bad" commit (where the bug exists).
- Git checks out intermediate commits, prompting you (or an automated test script) to classify each commit as `good` or `bad`, rapidly locating the root cause commit.

External Resources

- Go Wiki: Table-Driven Tests
- Git Documentation: git-bisect
- Cypress End-to-End Testing Guide

Go Test Layers

Use:

- Unit tests for config, validation, JWT, HFC scoring.
- Repository tests for SQL queries.
- Handler tests for HTTP status and JSON shape.
- Integration tests for full workflows.

Handler Tests

Use `httptest`:

```
req := httptest.NewRequest(http.MethodGet, "/api/updates", nil)
rr := httptest.NewRecorder()
handler.ServeHTTP(rr, req)
```

Assert:

- status code
- content type
- JSON fields
- error shape

Compatibility Tests

For each endpoint, test:

- method
- path
- auth requirement
- request payload
- response shape
- important business rules

Examples:

- Registration closed returns 403.
- Blocked user cannot login.
- Grant submission requires whitelist when enabled.
- Non-approver cannot approve grant.
- HFC shadow mode does not block approval.

Vue Testing

Use component tests for:

- route guard behavior
- login form submission
- language switching
- grant form conditional sections
- admin table filters

Use end-to-end tests for:

- login to dashboard
- create/update business profile
- admin login and report loading
- grant application submission path

Debugging Tools

Backend:

```
go test ./...
go test ./internal/grant -run TestGrantAccess -v
go test -race ./...
```

Frontend:

```
npm run test
npm run build
```

Git:

```
git diff
git blame path/to/file
git bisect start
```

Practical Examples

Example 1: Table-Driven Unit Test in Go

This complete testing code demonstrates table-driven unit testing using Go subtests for a business registration validation helper:

```
// File: internal/business/dto_test.go
package business

import (
■"testing"
)

func TestCreateBusinessRequest_Validate(t *testing.T) {
■// Table of test cases
■tests := []struct {
■■name        string
■■request     CreateBusinessRequest
■■expectError bool
■■errorMsg    string
■}{
■■{
■■■name: "Valid request",
■■■request: CreateBusinessRequest{
■■■■Name:            "Khyber Tech Solutions",
■■■■Address:         "Main Bazaar, Swat",
■■■■District:        "Swat",
■■■■MaleEmployees:   5,
```

```
    FemaleEmployees: 2,
  },
  expectError: false,
 },
 {
  name: "Empty business name",
  request: CreateBusinessRequest{
   Name:            "",
   Address:         "Main Bazaar, Swat",
   District:        "Swat",
   MaleEmployees:   1,
   FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg:    "business name is required",
 },
 {
  name: "Invalid district out of scope",
  request: CreateBusinessRequest{
   Name:            "Khyber Tech Solutions",
   Address:         "Main Bazaar, Swat",
   District:        "Peshawar", // not in allowed list
   MaleEmployees:   1,
   FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg:    "business location district is out of allowed scope",
 },
 {
  name: "Negative employee counts",
  request: CreateBusinessRequest{
   Name:            "Khyber Tech Solutions",
   Address:         "Main Bazaar, Swat",
   District:        "Swat",
   MaleEmployees:   -1,
   FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg:    "employee counts cannot be negative",
 },
}

 // Iterate over the table
 for _, tt := range tests {
  t.Run(tt.name, func(t *testing.T) {
   err := tt.request.Validate()

   if tt.expectError {
    if err == nil {
     t.Errorf("expected error containing %q, got nil", tt.errorMsg)
    } else if err.Error() != tt.errorMsg {
     t.Errorf("expected error message %q, got %q", tt.errorMsg, err.Error())
    }
   } else {
    if err != nil {
     t.Errorf("unexpected error: %v", err)
    }
   }
  })
 }
}
```

Example 2: Automatic Git Bisect Session

To find a commit that broke a test automatically:

```
# Start bisect session
git bisect start

# Mark current HEAD as bad
git bisect bad

# Mark a known good tag or commit hash (e.g., release-v1.0.0)
git bisect good release-v1.0.0

# Run git bisect automatically using a test script
git bisect run go test ./internal/business -run TestCreateBusinessRequest_Validate
```

Git will iterate through the history, run the test, and output the first bad commit.

Mastery Check

You understand this chapter when you can:

- Write table-driven Go tests.
- Test authenticated HTTP handlers.
- Detect API contract regressions.
- Reproduce a bug with a failing test before fixing it.
- Use Git history to find when behavior changed.

Chapter 16: Deployment, Release Git, and Mastery Roadmap

Purpose

The final stage is operating the rewritten application: build artifacts, containers, migrations, environment configuration, release tags, and rollback strategy.

Theoretical Background

Multi-Stage Container Builds

Docker images are built layer-by-layer. Every instruction in a `Dockerfile` (e.g., `RUN`, `COPY`) adds a new layer to the image.

- **Cache Optimization:** By copying dependency files (like `go.mod` and `go.sum`) first and running `go mod download`, Docker caches the dependencies. Source code changes will not trigger re-downloading dependencies, speeding up builds.
- **Multi-Stage Builds:** Allow you to separate compile environments from deployment runtimes. You compile the Go app inside a heavy, tool-rich image (`golang:1.22`), then copy only the compiled binary into a minimal execution runtime image (like `alpine` or `scratch`).
- This reduces the final production image size from ~1GB down to ~30MB, eliminating build tools and reducing the security attack surface.

```
[ Stage 1: Build Env ] ---> Compiles Go code ---> (Server Binary)
                                                        | (Copy Binary)
                                                        v
[ Stage 2: Runtime Env ] <----------------------- [ Alpine / Scratch Image ] (Minimal Size)
```

Semantic Versioning (SemVer)

Releases are versioned using a standard three-part format: `MAJOR.MINOR.PATCH` (e.g., `v1.4.2`):

- **MAJOR:** Increment when you make incompatible API changes.
- **MINOR:** Increment when you add functionality in a backwards-compatible manner.
- **PATCH:** Increment when you make backwards-compatible bug fixes.

Git Flow Branching Model

A structured branching workflow facilitates organized software development and releases:

- `main`**:** Stores production-ready code. Every commit here matches a tagged release.
- `develop`**:** Integration branch for features.
- `feature/` **branches:** Short-lived branches created from `develop` to work on single features, merged back via pull requests.
- `release/` **branches:** Created from `develop` when preparing for a release. Only bug fixes are allowed here before merging into `main` and `develop`.
- `hotfix/` **branches:** Branches cut directly from `main` to address critical production issues, merged back to `main` and `develop`.

External Resources

- Semantic Versioning 2.0.0 Specification
- Docker Documentation: Best Practices for Writing Dockerfiles
- Atlassian Git Tutorials: Git Flow Workflows

Deployment Units

The system has:

- PostgreSQL 15.
- Redis 7.
- Go backend.
- Go worker or compatible worker process.
- Vue frontend served by nginx.
- Adminer or pgAdmin for database inspection.

Release Git

Use tags:

```
git tag -a v0.1.0 -m "First Go API compatibility milestone"
git push origin v0.1.0
```

Use release branches when stabilizing:

```
git checkout -b release/v1.0.0
```

Use hotfix branches from the release tag:

```
git checkout -b hotfix/login-blocked-users v1.0.0
```

Rollback Strategy

A rollback plan includes:

- Previous container image tag.
- Database migration rollback or forward-fix plan.
- Preserved environment variables.
- Redis cache invalidation.
- Smoke tests after rollback.

Never deploy schema changes casually. Database compatibility is harder to roll back than code.

Practical Examples

Example 1: Multi-Stage Production Dockerfile for Go Backend

This Dockerfile compiles a statically-linked Go binary in a build stage and runs it inside a minimal Alpine container containing no compilers or build tools:

```
# STEP 1: Build phase
FROM golang:1.22-alpine AS builder

# Install certificates and git
RUN apk update && apk add --no-cache git ca-certificates

WORKDIR /app

# Copy dependency manifests first to leverage Docker caching
COPY go.mod go.sum ./
```

```
RUN go mod download

# Copy application source files
COPY . .

# Compile binary statically, disabling CGO and targetting OS/Arch
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags="-w -s" -o server ./cmd/server

# STEP 2: Minimal Runtime phase
FROM alpine:3.19

# Import certificates to enable secure outbound HTTPS calls
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app/server /usr/local/bin/server

# Create a non-root group and user for security isolation
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 8080

ENTRYPOINT ["/usr/local/bin/server"]
```

Example 2: Dockerfile and Nginx setup for Vue Frontend

This multi-stage Dockerfile builds the static Vue asset bundle and serves it using Nginx, including custom routing fallbacks for single page applications:

```
# STEP 1: Node Build phase
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# STEP 2: Nginx Web Server phase
FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html

# Replace default configuration with one optimized for SPA routers
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Example supporting config file (`nginx.conf`) to ensure client-side routing fallback:

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        # Direct all URI requests to index.html for client-side routing
        try_files $uri $uri/ /index.html;
    }

    # Proxy API calls to Go backend server
    location /api {
        proxy_pass http://sme_backend_app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Mastery Roadmap

To master Go:

- Read standard library HTTP code.

- Practice contexts, interfaces, errors, and tests.
- Profile slow report queries.
- Use `go test -race`.
- Refactor only after tests protect behavior.

To master Vue:

- Build small components with clear props and emits.
- Keep API calls centralized.
- Manage form state explicitly.
- Test route guards and complex conditional forms.
- Treat accessibility and bilingual layout as core requirements.

To master Git:

- Commit in small units.
- Read diffs before every commit.
- Use branches intentionally.
- Practice conflict resolution.
- Tag releases.
- Learn bisect for regression hunting.

Final Capstone

The capstone is complete when:

- All documented `/api` endpoints exist in Go.
- Existing Vue screens work against the Go backend.
- New Vue screens reproduce the applicant and admin workflows.
- PostgreSQL schema matches the original.
- Redis caching and access control work.
- HFC scoring and approval logs work.
- S3 uploads work.
- Email jobs are queued.
- Tests cover critical workflows.
- A release tag marks the milestone.

Chapter 17: Go Beginner Primer Through Portal Features

Purpose

This chapter is the missing beginner bridge. If you are new to Go, do not start by memorizing syntax. Start by asking: "Where will this Go concept appear in the PEACE SME portal?" Every concept below is paired with a real feature you will build.

How Go Feels Different From Python or JavaScript

Go is compiled, statically typed, explicit, and intentionally small. That means:

- The compiler checks many mistakes before the program runs.
- You must say what shape data has.
- Errors are returned as values instead of thrown as exceptions.
- Packages are simple directories.
- Concurrency is built into the language through goroutines and channels, but web applications still need careful design.
- Formatting is standardized with `gofmt`.

In Flask, it is common to pass dictionaries around. In Go, you usually define structs. In JavaScript, a field can silently change type. In Go, `GrantRequired float64` cannot suddenly become `"five hundred thousand"` unless you explicitly parse it.

Concept Map

Go concept	Portal task	Why it matters
Variables and constants	allowed districts, status names, cache TTLs	Avoid magic strings scattered through handlers
Structs	`User`, `Business`, `Grant`, `AdminUser`	Model request, response, and database shapes
Pointers	services, repositories, large structs	Share dependencies without copying them
Methods	`UserService.Login`, `GrantRepository.FindByUserID`	Attach behavior to a type
Interfaces	`EmailQueue`, `Storage`, `TokenSigner`	Swap real services for test fakes
Slices	lists of documents, financed items, admins	Represent ordered collections
Maps	allowed country codes, translations, filter allow-lists	Fast lookup by key
Errors	config parsing, SQL failures, validation	Make failure explicit and testable
Context	request cancellation, identity, database queries	Carry request lifecycle through layers
Goroutines	background workers, email sending, HFC jobs	Run work outside the request path
Testing	auth rules, grant validation, reports	Protect compatibility while rewriting

Variables, Constants, and Types

Theory: A variable stores a value. A constant stores a value known at compile time. Go types are strict.

Application parallel: allowed districts should be constants or a package-level list:

```
var AllowedDistricts = map[string]bool{
    "Swat":          true,
    "Shangla":       true,
    "Upper Dir":     true,
    "Upper Chitral": true,
    "Lower Chitral": true,
}
```

Beginner rule: use constants for stable single values and maps/slices for groups.

```
const StatusBlocked = "blocked"
const StatusUnblocked = "unblocked"
```

Project task: create `internal/domain/constants.go` and put user statuses, grant statuses, HFC statuses, risk levels, and allowed districts there.

Git task:

```
git checkout -b chapter-17-domain-constants
git add internal/domain
git commit -m "Add domain constants for portal workflow"
```

Structs

Theory: A struct groups named fields into one type. This is how Go represents application data.

Application parallel: the login response has a specific shape:

```
{ "token": "jwt", "user_id": 42, "language": "urdu" }
```

Model it:

```
type LoginResponse struct {
    Token    string `json:"token"`
    UserID   int64  `json:"user_id"`
    Language string `json:"language"`
}
```

The JSON tags are not optional decoration. They preserve the existing frontend contract. Without `json:"user_id"`, Go would output `UserID`, which would break code expecting `user_id`.

Project task: define request and response structs for:

- user login
- admin login
- business create/update
- grant create/update
- paginated report responses

Beginner mistake to avoid: do not use one giant struct for everything. A database row, a create request, and an API response often have different fields.

Pointers

Theory: A pointer stores the address of a value. Go passes function arguments by value, so without pointers you copy values.

Application parallel: services hold dependencies:

```
type UserService struct {
    users  *UserRepository
    tokens *TokenService
}
```

You pass pointers because:

- repositories may contain a database pool;

- services should share the same dependency instance;
- methods can avoid copying large values.

Project task: create constructors:

```
func NewUserService(users *UserRepository, tokens *TokenService) *UserService {
    return &UserService{users: users, tokens: tokens}
}
```

Beginner rule: use values for small immutable data. Use pointers for services, repositories, large structs, and anything that must be modified.

Methods

Theory: A method is a function attached to a type.

Application parallel: instead of a free function:

```
func Login(service UserService, email string, password string) {}
```

write:

```
func (s *UserService) Login(ctx context.Context, req LoginRequest) (*LoginResponse, error) {
    // login behavior
}
```

This reads naturally: "the user service logs in a user."

Project task: implement methods for:

- `UserService.Login`
- `BusinessService.SaveProfile`
- `GrantService.Apply`
- `AdminService.SetUserStatus`
- `HFCService.CalculateForUser`

Interfaces

Theory: An interface describes behavior. In Go, a type satisfies an interface automatically if it has the required methods.

Application parallel: grant approval must enqueue emails. In tests, you do not want to send real Brevo emails.

```
type EmailQueue interface {
    EnqueueGrantApproved(ctx context.Context, userID int64) error
    EnqueueApprovalNotification(ctx context.Context, userID int64) error
}
```

The real implementation writes to Redis. The test implementation records calls in memory.

Project task: define interfaces at the package that uses them. If `grant.Service` needs email queue behavior, define the small interface in `internal/grant`, not a giant global interface package.

Beginner rule: accept interfaces, return concrete types.

Slices

Theory: A slice is a dynamic list.

Application parallel: a grant has financed items:

```
type FinancedItem struct {
    Item          string  `json:"item"`
    Quantity      int     `json:"quantity"`
    EstimatedCost float64 `json:"estimated_cost"`
```

```
}

type GrantRequest struct {
    FinancedItems []FinancedItem `json:"financed_items"`
}
```

Project task: validate that each financed item has a name, positive quantity, and positive cost.

Maps

Theory: A map is a key-value lookup table.

Application parallel: report sorting must be safe. Never put raw `sort_by` directly into SQL.

```
var applicantSortColumns = map[string]string{
    "created_at": "u.created_at",
    "district":   "b.business_location_district",
    "status":     "u.status",
}
```

Project task: use map allow-lists for:

- report sort columns
- allowed countries
- allowed districts
- admin roles
- HFC risk bands if helpful

Errors

Theory: Go functions return errors. You must check them.

Application parallel: login can fail because the request body is invalid, the user is missing, the password is wrong, the user is blocked, or token signing failed.

Do not collapse all errors too early. Inside the service, keep meaningful errors:

```
var ErrInvalidCredentials = errors.New("invalid credentials")
var ErrUserBlocked = errors.New("user is blocked")
```

The handler converts them into HTTP responses:

```
switch {
case errors.Is(err, ErrInvalidCredentials):
    httpx.WriteError(w, http.StatusUnauthorized, "invalid_credentials", "Invalid email or
password")
case errors.Is(err, ErrUserBlocked):
    httpx.WriteError(w, http.StatusForbidden, "user_blocked", "Your account is blocked")
default:
    httpx.WriteError(w, http.StatusInternalServerError, "server_error", "Unexpected error")
}
```

Project task: define service-level errors for auth, validation, not found, forbidden, and conflict cases.

Context

Theory: `context.Context` carries cancellation, timeouts, and request-scoped values.

Application parallel: when a browser cancels a report request, the Go handler's context is canceled. Database queries should receive that context so PostgreSQL work can stop too.

```
func (r *ReportRepository) Applicants(ctx context.Context, filter ApplicantFilter) ([
]ApplicantRow, error) {
    rows, err := r.db.Query(ctx, query, args...)
}
```

Project task: every handler should call service methods with `r.Context()`. Every repository method should accept `ctx context.Context`.

Goroutines and Background Work

Theory: a goroutine is a lightweight concurrent function started with `go`.

Application parallel: HFC scoring and email sending should not block the applicant's grant submission response.

Beginner warning: do not casually start goroutines inside handlers for production-critical work. If the process crashes, the goroutine disappears. Use Redis-backed jobs or an outbox table for reliable work.

Good learning progression:

1. Start with synchronous fake implementations.
2. Add interfaces.
3. Add Redis queue implementation.
4. Add worker process.
5. Add retry and logging.

Testing

Theory: Go tests live in files ending with `_test.go`.

Application parallel: write table-driven tests for grant validation:

```
func TestValidateGrantRequest(t *testing.T) {
    tests := []struct {
        name    string
        req     GrantRequest
        wantErr bool
    }{
        {name: "missing amount", req: GrantRequest{}, wantErr: true},
        {name: "valid financed item", req: validGrant(), wantErr: false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := ValidateGrantRequest(tt.req)
            if (err != nil) != tt.wantErr {
                t.Fatalf("expected error=%v, got %v", tt.wantErr, err)
            }
        })
    }
}
```

Project task: create tests for:

- configuration parsing
- JWT signing and verification
- blocked user login
- grant whitelist gate
- HFC risk scoring
- report sort allow-list

Capstone Exercise

Build a tiny vertical slice:

1. Define `LoginRequest` and `LoginResponse`.

2. Implement `UserRepository.FindByEmail`.
3. Implement `UserService.Login`.
4. Implement `POST /api/login`.
5. Add a Vue login form that stores `userToken`.
6. Commit each layer separately.

When this works, you have practiced structs, methods, pointers, interfaces, errors, context, JSON, SQL, Vue state, and Git.

Chapter 18: Project Parallel Workbook

Purpose

This chapter turns the book into a build plan. Each row pairs a concept with a concrete PEACE SME implementation task. Use it when you ask, "What should I build to understand this topic?"

Parallel Task Table

Learning topic	Theory to study	Backend task	Vue task	Git task
HTTP routing	request methods, paths, status codes	implement `/health` and `/api/updates`	show updates on landing page	commit route and handler separately
JSON encoding	struct tags, request decoding	implement `ReadJSON` and `WriteJSON`	consume JSON with Axios	inspect diff for response names
Config	env vars, defaults, validation	parse `GRANT_APPLICATION_OPEN`	show registration closed page	add config tests before commit
Auth	JWT, bcrypt, bearer headers	implement `/api/login`	store `userToken`	branch `feature/user-login`
Admin auth	roles and permissions	implement `/api/admin/login`	store `adminToken`	commit admin claims tests
Middleware	chain of handlers	auth, geo-block, access slot middleware	redirect unauthenticated routes	diff middleware order
Database	schema, indexes, joins	migrate `users`, `businesses`	dashboard fetches profile	tag first DB milestone
Transactions	atomic multi-step writes	approve grant and log action	approval button	commit failing test then fix
JSONB	typed nested data	save `financed_items`	dynamic item rows	review JSON names
Validation	business rule enforcement	allowed districts, contribution rules	inline form errors	commit validation table tests
Uploads	presigned URLs, object keys	generate upload URL	document upload component	branch `feature/uploads`
Redis	TTL, cache, debounce	cache FAQs and updates	FAQ drawer consumes API	commit cache invalidation
Reports	pagination, filters, sorting	applicant report endpoint	admin table filters	compare query params diff
CSV	streaming, query tokens	full applicant export	export button	add token expiry test
HFC	deterministic scoring	calculate score and risk	HFC queue display	commit rule tests one by one
Workers	durable background jobs	enqueue HFC and email jobs	show pending state	tag async milestone
Bilingual UI	translation dictionaries, RTL	return language from login	language switcher	commit English/Urdu together
Deployment	containers, migrations	Dockerfile and migration runner	static frontend build	tag release candidate

Milestone 1: Skeleton That Teaches Go Basics

Goal: make the Go program feel understandable.

Build:

- `cmd/server/main.go`
- `internal/config`
- `internal/httpx`
- `internal/health`

- `internal/app`

Theory:

- packages
- exported vs unexported names
- functions
- structs
- pointers
- errors

Portal feature:

- `/health` proves the server runs.
- `/api/updates` proves a public JSON route works.

Vue parallel:

- create a landing page section that calls `/api/updates`.

Git:

```
git checkout -b milestone-01-server-skeleton
git commit -m "Add Go server skeleton"
git commit -m "Add public updates endpoint"
```

Done when:

- `go test ./...` passes.
- `/health` returns JSON.
- `/api/updates` returns an empty list or seeded rows.

Milestone 2: Login Vertical Slice

Goal: understand web auth end to end.

Build:

- `users` table migration.
- `UserRepository.FindByEmail`.
- bcrypt password verification.
- JWT token generation.
- `POST /api/login`.
- Vue login page.
- route guard for `/dashboard`.

Theory:

- SQL query result scanning.
- nullable values.
- service errors.
- JWT claims.
- localStorage.

- Axios headers.

Portal feature:

- applicant logs in with email and password.
- blocked user receives a specific failure.
- successful login returns `{token, user_id, language}`.

Git:

```
git checkout -b feature/user-login
git commit -m "Add user login repository"
git commit -m "Add JWT user token service"
git commit -m "Implement user login endpoint"
git commit -m "Add Vue user login flow"
```

Done when:

- invalid password fails.
- blocked user fails.
- valid login stores `userToken`.
- authenticated route sends `Authorization: Bearer`.

Milestone 3: Business Profile

Goal: learn create/update workflows and validation.

Build:

- `businesses` migration.
- `BusinessRequest` struct.
- allowed district validation.
- `GET /api/business`.
- `POST /api/business`.
- `PUT /api/business`.
- Vue business profile form.

Theory:

- one-to-one table relation.
- unique constraints.
- validation before database writes.
- form state with `reactive`.

Portal feature:

- each applicant has one business profile.
- district must be in scope.
- profile can be edited.

Git:

```
git checkout -b feature/business-profile
git commit -m "Add business profile schema"
git commit -m "Implement business profile service"
git commit -m "Add Vue business profile form"
```

Done when:

- creating twice does not create duplicate rows.
- invalid district is rejected.
- form loads existing profile.

Milestone 4: Grant Form and Whitelist Gate

Goal: master larger payloads and workflow rules.

Build:

- `grants` migration.
- `grant_access_whitelist` migration.
- grant request validation.
- `GET /api/grant`.
- `POST /api/grant`.
- `PUT /api/grant`.
- `GET /api/grant-status`.
- Vue multi-section grant form.

Theory:

- JSONB fields.
- slices of structs.
- conditional validation.
- workflow gates.
- HTTP 403 vs 409 vs 422.

Portal feature:

- user can apply only when whitelisted if `GRANT_REQUIRE_SELECTION=1`.
- one user has one grant.
- financed items repeat dynamically.
- SRSP relatives are conditional rows.

Git:

```
git checkout -b feature/grant-application
git commit -m "Add grant schema and models"
git commit -m "Implement grant access whitelist"
git commit -m "Implement grant application endpoint"
git commit -m "Add Vue grant application form"
```

Done when:

- non-whitelisted user cannot submit.
- whitelisted user can submit.
- JSONB fields round-trip.
- Vue review page displays the same data.

Milestone 5: Admin Reports

Goal: learn real data querying.

Build:

- admin token verification.
- applicant report endpoint.
- filters, sort allow-list, pagination.
- admin report table.
- CSV export query token.

Theory:

- joins.
- count queries.
- pagination.
- SQL injection prevention.
- streaming responses.

Portal feature:

- admins can filter applicants by district, sector, status, language, gender, and search.

Git:

```
git checkout -b feature/admin-reports
git commit -m "Add applicant report query"
git commit -m "Add report filters and pagination"
git commit -m "Add Vue admin applicant report"
```

Done when:

- unauthorized users cannot access reports.
- invalid sort field is rejected or replaced by a safe default.
- pagination response shape is exactly `{data,total,page,per_page}`.

Milestone 6: HFC and Approval

Goal: learn deterministic business logic, auditability, and side effects.

Build:

- HFC scoring rules.
- `hfc_evaluations` writes.
- admin HFC queue.
- approval check endpoint.
- approval transaction.
- email enqueue interface.

Theory:

- pure functions.
- risk bands.
- audit logs.
- transactions.

- interfaces for side effects.

Portal feature:

- grant submission triggers HFC scoring.
- approver can approve grants.
- shadow mode means HFC does not block approval.

Git:

```
git checkout -b feature/hfc-approval
git commit -m "Add deterministic HFC scoring"
git commit -m "Store HFC evaluations"
git commit -m "Implement grant approval transaction"
```

Done when:

- scoring tests prove each rule.
- approval writes both grant and log.
- non-approver cannot approve.
- shadow mode behavior is tested.

Milestone 7: Production Readiness

Goal: make it runnable and maintainable.

Build:

- Dockerfile.
- migration runner.
- Redis cache.
- worker process.
- S3 upload integration.
- structured logs.
- smoke tests.

Theory:

- deployment units.
- process lifecycle.
- graceful shutdown.
- observability.
- rollback.

Portal feature:

- full applicant workflow and admin workflow run on the Go + Vue stack.

Git:

```
git checkout -b release/v0.1.0
git tag -a v0.1.0 -m "First Go and Vue portal milestone"
```

Done when:

- backend starts from clean environment.

- migrations apply once.
- Vue build succeeds.
- critical tests pass.
- release tag exists.

Chapter 19: Vue and Git Practice Lab

Purpose

This lab makes Vue and Git practical. You will build frontend behavior that matches the Go API while using Git to protect every step.

Vue Beginner Concepts by Portal Feature

Vue concept	Portal feature	What you learn
`ref`	login email/password fields	single primitive reactive values
`reactive`	business profile form	object-shaped form state
`computed`	Urdu/English label selection	derived state
`watch`	filters triggering report reloads	reacting to changes
`onMounted`	fetch dashboard data	lifecycle fetching
props	reusable form input	parent-to-child data
emits	modal close, upload complete	child-to-parent events
router guards	protected dashboard/admin pages	client-side auth flow
composables	`useAuth`, `useLanguage`, `usePagination`	reusable stateful logic
Axios interceptors	bearer auth headers	API integration

Lab 1: Login Page

Theory:

- `ref` is best for simple values.
- `v-model` binds input value to state.
- submit handlers call API functions.

Portal task:

- build `UserLogin.vue`.
- call `POST /api/login`.
- store `userToken`.
- redirect to `/dashboard`.

Example:

```
<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import api from '../api/client'

const router = useRouter()
const email = ref('')
const password = ref('')
const error = ref('')
const loading = ref(false)

async function login() {
  error.value = ''
  loading.value = true
  try {
    const { data } = await api.post('/login', {
      email_address: email.value,
```

```
      password: password.value,
    })
    localStorage.setItem('userToken', data.token)
    router.push('/dashboard')
  } catch (err) {
    error.value = 'Invalid login or blocked account'
  } finally {
    loading.value = false
  }
}
</script>
```

Git practice:

```
git checkout -b vue-user-login
git add frontend/src/views/UserLogin.vue
git commit -m "Add Vue user login page"
```

Lab 2: Business Profile Form

Theory:

- `reactive` is useful for form objects.
- select inputs should store backend-compatible values.
- frontend validation improves UX, but backend validation remains authoritative.

Portal task:

- build `SmeBusinessProfile.vue`.
- fetch existing data with `GET /api/business`.
- create with `POST /api/business`.
- update with `PUT /api/business`.

State shape:

```
const form = reactive({
  name_of_business: '',
  business_registration_number: '',
  business_location_district: '',
  business_sector: '',
  male_employees: 0,
  female_employees: 0,
})
```

Parallel Go task:

- `BusinessRequest` struct has matching JSON tags.
- Go validates allowed districts.
- repository uses `INSERT` or `UPDATE`.

Git practice:

```
git checkout -b vue-business-profile
git commit -m "Add business profile form state"
git commit -m "Connect business profile form to API"
```

Lab 3: Grant Application Dynamic Sections

Theory:

- arrays in Vue are reactive when stored inside `reactive`.
- conditional rendering uses `v-if`.
- repeated rows use `v-for`.

Portal task:

- add financed item rows.
- show cash fields only when contribution includes cash.
- show in-kind fields only when contribution includes in-kind.
- show relatives table only when `has_srsp_relative` is true.

Example:

```
function addFinancedItem() {
  form.financed_items.push({
    item: '',
    quantity: 1,
    estimated_cost: null,
  })
}
```

Parallel Go task:

- decode `financed_items` into `[]FinancedItem`.
- validate each row.
- store as JSONB.

Git practice:

```
git checkout -b vue-grant-form
git commit -m "Add grant financed items UI"
git commit -m "Add conditional grant contribution fields"
git commit -m "Connect grant form to API"
```

Lab 4: Admin Report Table

Theory:

- tables need query state: page, per page, filters, sort.
- `watch` can reload data when filters change.
- query params should mirror backend filter names.

Portal task:

- build applicant report table.
- support `page`, `per_page`, `district`, `sector`, `status`, `search`, `sort_by`, `sort_dir`.

Parallel Go task:

- parse query params into `ApplicantReportFilter`.
- use SQL allow-list for sort fields.
- return `{data,total,page,per_page}`.

Git practice:

```
git checkout -b vue-admin-report
git commit -m "Add admin applicant report table"
git commit -m "Connect admin report filters"
```

Git Lab: Learn by Reviewing Your Own Work

Before every commit:

```
git status
git diff
git add <files>
```

```
git diff --staged
git commit -m "Specific behavior"
```

Review questions:

- Did I include unrelated files?
- Did I accidentally commit generated output?
- Does the commit message describe behavior?
- Could this commit be reverted safely?
- Are tests or screenshots needed?

Git Lab: Feature Branch to Merge

```
git checkout main
git checkout -b feature/grant-form
# build the feature
git add frontend/src/views/SmeGrantApplication.vue
git commit -m "Add grant application form"
git checkout main
git merge feature/grant-form
```

Conflict practice:

1. Create two branches that edit the same route list.
2. Merge one branch.
3. Merge the second branch.
4. Resolve the route conflict manually.
5. Run tests.
6. Commit the merge.

Capstone Lab

Build one complete user story:

1. User logs in.
2. User opens dashboard.
3. User creates business profile.
4. Admin marks applicant eligible.
5. Admin whitelists user for grants.
6. User submits grant.
7. HFC score appears for admin.
8. Approver approves grant.

For every step, write down:

- Vue route.
- API endpoint.
- Go handler.
- Go service method.
- Repository query.
- Database table.

- Git commit.

This is the real learning loop. The concepts become durable when you can trace a button click all the way to SQL and back.